

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng trò chơi trực tuyến nhiều người chơi thể
loại thẻ tướng chiến thuật tích hợp hệ thống xếp hạng
thời gian thực**

BÙI XUÂN HẢI

hai.bx194268@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Tưởng Văn Đạt

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng trò chơi trực tuyến nhiều người chơi thể
loại thẻ tướng chiến thuật tích hợp hệ thống xếp hạng
thời gian thực**

BÙI XUÂN HẢI

hai.bx194268@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Tưởng Văn Đạt _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn chân thành đến quý thầy cô Trường Công nghệ Thông tin và Truyền thông, đặc biệt là thầy PGS. TS. **Tưởng Văn Đạt** đã tận tình chỉ bảo, định hướng cho em trong suốt quá trình thực hiện đồ án.

Con cảm ơn gia đình đã luôn đồng hành và là chỗ dựa vững chắc nhất. Cảm ơn người yêu cùng bạn bè vì đã kiên nhẫn lắng nghe, chia sẻ áp lực và động viên mình đi qua những ngày tháng khó khăn.

Cuối cùng, tôi muốn gửi lời cảm ơn đến chính bản thân mình. Cảm ơn vì đã kiên trì, chăm chỉ và giữ vững quyết tâm vượt qua mọi thử thách để hoàn thành đồ án này với kết quả tốt nhất.

TÓM TẮT ĐỒ ÁN TỐT NGHIỆP

Trong bối cảnh ngành công nghiệp trò chơi trực tuyến phát triển mạnh mẽ, việc xây dựng hệ thống máy chủ backend hỗ trợ trò chơi nhiều người chơi thời gian thực với hiệu năng cao, độ trễ thấp và khả năng mở rộng tốt là một bài toán vô cùng quan trọng. Các giải pháp truyền thống dựa trên kiến trúc nguyên khối thường gặp khó khăn về khả năng mở rộng và đồng bộ dữ liệu tải cao. Đồ án này lựa chọn hướng tiếp cận tự phát triển hệ thống backend theo kiến trúc microservices sử dụng ngôn ngữ Go kết hợp giao thức truyền thông gRPC/Protobuf. Lựa chọn này giúp tận dụng tối đa hiệu suất xử lý song song vượt trội của Go qua Goroutines và tối ưu hóa băng thông mạng nhờ gRPC. Giải pháp đề xuất bao gồm ứng dụng client xây dựng trên Unity kết nối qua Gateway đến các dịch vụ backend độc lập: Login-server xử lý xác thực, World-server quản lý thông tin đệ tử, xây dựng tông môn, tính toán điểm lực chiến và cập nhật bảng xếp hạng thời gian thực, cùng Combat-server phụ trách mô phỏng các trận đấu chiến thuật. Hệ thống sử dụng PostgreSQL để lưu trữ dữ liệu bền vững và tích hợp Redis để quản lý phiên đăng nhập nhanh chóng. Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định dưới tải cao, thời gian phản hồi nhanh, độ trễ thấp và đảm bảo tính chính xác khi cập nhật bảng xếp hạng. Đóng góp chính của đồ án là thiết kế và triển khai thành công một kiến trúc máy chủ game microservices hoàn chỉnh, có hiệu năng tốt và khả năng mở rộng linh hoạt cho các tựa game chiến thuật trực tuyến.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.2 Tổng quan chức năng	5
2.2.1 Biểu đồ use case tổng quát	5
2.2.2 Biểu đồ use case phân rã các phân hệ chính	5
2.2.3 Quy trình nghiệp vụ cốt lõi	6
2.3 Đặc tả chức năng	6
2.3.1 Đặc tả use case Đăng nhập hệ thống	6
2.3.2 Đặc tả use case Xây dựng và nâng cấp công trình.....	7
2.3.3 Đặc tả use case Chiêu mộ đệ tử	7
2.3.4 Đặc tả use case Vượt ải chiến dịch	8
2.3.5 Đặc tả use case Xem bảng xếp hạng thời gian thực	8
2.3.6 Đặc tả use case Tự động điều phối vùng máy chủ khi quá tải (Roll Server)	9
2.4 Yêu cầu phi chức năng	10
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	12
3.1 Ngôn ngữ lập trình Go cho hệ thống máy chủ.....	12
3.1.1 Đặc điểm nổi bật.....	12
3.1.2 Giải pháp thay thế và Lý do lựa chọn	12

3.2 Giao thức truyền thông gRPC và Protocol Buffers	13
3.2.1 Đặc điểm nổi bật.....	13
3.2.2 Giải pháp thay thế và Lý do lựa chọn	14
3.3 Hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL.....	14
3.3.1 Đặc điểm nổi bật.....	14
3.3.2 Giải pháp thay thế và Lý do lựa chọn	14
3.4 Hệ thống lưu trữ dữ liệu Redis	15
3.4.1 Đặc điểm nổi bật.....	15
3.4.2 Giải pháp thay thế và Lý do lựa chọn	15
3.5 Game Engine Unity 6 cho ứng dụng Client	16
3.5.1 Đặc điểm nổi bật.....	16
3.5.2 Giải pháp thay thế và Lý do lựa chọn	16
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	17
4.1 Thiết kế kiến trúc.....	17
4.1.1 Lựa chọn kiến trúc phần mềm	17
4.1.2 Thiết kế tổng quan.....	18
4.1.3 Thiết kế chi tiết gói	20
4.2 Thiết kế chi tiết.....	21
4.2.1 Thiết kế giao diện	21
4.2.2 Thiết kế lớp	22
4.2.3 Thiết kế cơ sở dữ liệu	26
4.3 Xây dựng ứng dụng.....	29
4.3.1 Thư viện và công cụ sử dụng	29
4.3.2 Kết quả đạt được	30
4.3.3 Minh họa các chức năng chính	31

4.4 Kiểm thử.....	32
4.5 Triển khai	34
4.5.1 Mô hình triển khai hệ thống	34
4.5.2 Cấu hình mạng và Cân bằng tải (Reverse Proxy)	35
4.5.3 Cơ chế Roll Server điều phối phân vùng.....	35
4.5.4 Đánh giá kết quả triển khai thử nghiệm	36
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	38
5.1 Thiết kế giao diện thích ứng (Responsive UI) và quản lý phông chữ trong Unity 6.....	38
5.1.1 Dẫn dắt và giới thiệu bài toán	38
5.1.2 Giải pháp đề xuất	39
5.1.3 Kết quả đạt được	40
5.2 Cơ chế xử lý đồng thời và đồng bộ hóa tài nguyên nhàn rỗi (Idle Resource Generation) an toàn giao dịch.....	41
5.2.1 Dẫn dắt và giới thiệu bài toán	41
5.2.2 Giải pháp đề xuất	42
5.2.3 Kết quả đạt được	44
5.3 Giải pháp kiểm thử và xác thực nhật ký trận đấu (Combat Log Validation) chống gian lận hiệu năng cao	44
5.3.1 Dẫn dắt và giới thiệu bài toán	44
5.3.2 Giải pháp đề xuất	45
5.3.3 Kết quả đạt được	46
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	48
6.1 Kết luận.....	48
6.1.1 So sánh với các nghiên cứu hoặc sản phẩm tương tự.....	48
6.1.2 Phân tích quá trình thực hiện đồ án tốt nghiệp.....	51

6.2 Hướng phát triển.....	54
6.2.1 Các công việc cần thiết để hoàn thiện sản phẩm.....	54
6.2.2 Định hướng phát triển và nâng cấp mới	55
TÀI LIỆU THAM KHẢO.....	59
PHỤ LỤC.....	61
A. ĐẶC TẢ USE CASE.....	61
A.1 Đặc tả use case Đăng ký tài khoản.....	61
A.2 Đặc tả use case Sắp xếp đội hình chiến đấu	62
A.3 Đặc tả use case Thu thập tài nguyên công trình.....	62

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	5
Hình 4.1	Biểu đồ gói UML thể hiện sự phụ thuộc và giao tiếp trong hệ thống	19
Hình 4.2	Biểu đồ quan hệ giữa các thành phần trong gói World Server .	20
Hình 4.3	Biểu đồ trình tự chức năng Đăng ký tài khoản	25
Hình 4.4	Biểu đồ trình tự chức năng Thu thập tài nguyên công trình . .	26
Hình 4.5	Sơ đồ thực thể liên kết cơ sở dữ liệu (ERD) toàn hệ thống . . .	27
Hình 4.6	Mô hình triển khai máy chủ vi dịch vụ sử dụng Docker và Nginx	35

DANH MỤC BẢNG BIỂU

Bảng 3.1	Bảng so sánh lựa chọn công nghệ phát triển máy chủ backend	13
Bảng 4.1	Bảng chuẩn hóa phối màu giao diện người dùng	21
Bảng 4.2	Đặc tả thuộc tính và phương thức của lớp Player	23
Bảng 4.3	Đặc tả thuộc tính và phương thức của lớp PlayerBuilding . . .	24
Bảng 4.4	Đặc tả thuộc tính và phương thức của lớp PlayerHero	24
Bảng 4.5	Danh sách các thư viện và công cụ sử dụng trong dự án	30
Bảng 4.6	Thống kê số lượng mã nguồn và kích thước đóng gói sản phẩm	31
Bảng 4.7	Kịch bản kiểm thử chi tiết chức năng Đăng nhập	33
Bảng 4.8	Kịch bản kiểm thử chi tiết chức năng Thu thập tài nguyên . . .	33
Bảng 4.9	Kịch bản kiểm thử chi tiết chức năng Sắp xếp đội hình	34
Bảng 4.10	So sánh thông số hiệu năng hệ thống dưới các mức tải CCU khác nhau	37
Bảng 6.1	Bảng so sánh tổng quan giải pháp công nghệ của đề án với các sản phẩm tương tự	51

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
BaaS	Backend-as-a-Service - Mô hình dịch vụ đám mây cung cấp sẵn các chức năng phụ trợ phía máy chủ
CCU	Concurrent Users - Số lượng người dùng đồng thời tại một thời điểm
CPU	Central Processing Unit - Bộ xử lý trung tâm
ECDSA	Elliptic Curve Digital Signature Algorithm - Thuật toán ký số dựa trên mật mã đường cong Elliptic
ERD	Entity-Relationship Diagram - Sơ đồ mối quan hệ thực thể trong thiết kế cơ sở dữ liệu
gRPC	Google Remote Procedure Call - Giao thức truyền thông gọi thủ tục từ xa hiệu năng cao
HPA	Horizontal Pod Autoscaler - Cơ chế tự động co giãn pod theo chiều ngang trong Kubernetes
HTTP	Hypertext Transfer Protocol - Giao thức truyền tải siêu văn bản
HUST	Hanoi University of Science and Technology - Đại học Bách khoa Hà Nội
IP	Internet Protocol - Giao thức Internet
JSON	JavaScript Object Notation - Định dạng trao đổi dữ liệu mỏng nhẹ
JWT	JSON Web Token - Định dạng chuỗi mã hóa JSON dùng để xác thực phiên đăng nhập

Từ viết tắt	Ý nghĩa
K8s	Kubernetes - Nền tảng nguồn mở để tự động hóa việc triển khai, co giãn và quản lý các ứng dụng container
LTS	Long-Term Support - Phiên bản hỗ trợ dài hạn của phần mềm hoặc nền tảng
MAU	Monthly Active Users - Số lượng người dùng hoạt động hàng tháng
OCC	Optimistic Concurrency Control - Kiểm soát tương tranh lạc quan trong cơ sở dữ liệu
RAM	Random Access Memory - Bộ nhớ truy cập ngẫu nhiên
RDBMS	Relational Database Management System - Hệ quản trị cơ sở dữ liệu quan hệ
REST	Representational State Transfer - Mô hình kiến trúc thiết kế API dựa trên HTTP
SDF	Signed Distance Field - Trường khoảng cách có dấu, công nghệ dựng hình phong chữ vector
SoICT	School of Information and Communication Technology - Trường Công nghệ Thông tin và Truyền thông
SPoF	Single Point of Failure - Điểm nghẽn chịu lỗi duy nhất trong hệ thống
SQL	Structured Query Language - Ngôn ngữ truy vấn có cấu trúc
TCP	Transmission Control Protocol - Giao thức điều khiển truyền vận
TTL	Time To Live - Thời gian sống của dữ liệu hoặc khóa trong bộ nhớ đệm
UI	User Interface - Giao diện người dùng

Từ viết tắt	Ý nghĩa
UX	User Experience - Trải nghiệm người dùng
VPS	Virtual Private Server - Máy chủ riêng ảo

DANH MỤC THUẬT NGỮ

Thuật ngữ	Ý nghĩa
gRPC	Giao thức truyền thông gọi thủ tục từ xa mã nguồn mở hiệu năng cao dựa trên HTTP/2.
Protobuf	Định dạng mã hóa dữ liệu nhị phân nhỏ gọn, có cấu trúc và tối ưu của Google.
WebSocket	Giao thức hỗ trợ giao tiếp song công hai chiều thời gian thực giữa client và server qua một kết nối TCP.
Microservices	Kiến trúc thiết kế phần mềm bằng cách chia nhỏ hệ thống thành các dịch vụ độc lập.
Goroutine	Luồng thực thi siêu nhẹ (lightweight thread) được quản lý trực tiếp bởi Go runtime.
Redis	Hệ quản trị cơ sở dữ liệu lưu trữ trực tiếp trên RAM (in-memory) dùng để lưu cache và phiên đăng nhập.
PostgreSQL	Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, đáng tin cậy dùng để lưu trữ dữ liệu trò chơi.
Unity	Công cụ phát triển trò chơi đa nền tảng (game engine) được sử dụng để xây dựng phía Client.
API Gateway	Thành phần trung gian tiếp nhận, kiểm tra phân quyền và điều hướng các yêu cầu từ Client.
JWT	Định dạng chuỗi mã hóa JSON dùng làm chữ ký số xác thực phiên đăng nhập của người chơi.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày lý do lựa chọn đề tài, mục tiêu, phạm vi nghiên cứu và định hướng giải pháp của đề án tốt nghiệp "Xây dựng trò chơi trực tuyến nhiều người chơi thể loại thẻ tướng chiến thuật tích hợp hệ thống xếp hạng thời gian thực". Các nội dung chính bao gồm: đặt vấn đề về thách thức trong thiết kế hệ thống máy chủ trò chơi; xác định mục tiêu và phạm vi thực hiện; đề xuất giải pháp kỹ thuật; và giới thiệu bố cục của báo cáo đề án.

1.1 Đặt vấn đề

Trong những năm gần đây, sự phát triển của công nghệ thông tin và mạng Internet đã thúc đẩy ngành công nghiệp trò chơi trực tuyến trở thành một trong những lĩnh vực giải trí có tốc độ tăng trưởng nhanh nhất trên toàn cầu. Trong đó, thể loại trò chơi thẻ tướng chiến thuật thu hút lượng người chơi đông đảo nhờ tính toán chiến thuật cao và trải nghiệm tương tác phong phú. Người dùng hiện nay đòi hỏi trải nghiệm chơi game mượt mà, công bằng và có độ tin cậy cao. Điều này đặt ra thách thức lớn đối với hệ thống máy chủ backend, nơi xử lý toàn bộ logic trò chơi, đồng bộ trạng thái giữa các client và duy trì dữ liệu người chơi.

Thực tế phát triển hệ thống máy chủ trò chơi trực tuyến thời gian thực phải đối mặt với bài toán xử lý đồng thời phức tạp. Máy chủ cần đảm bảo độ trễ thấp khi truyền tải gói tin, đồng thời duy trì tính nhất quán của dữ liệu khi xảy ra hàng ngàn yêu cầu thay đổi tài sản trò chơi hoặc nâng cấp công trình trong cùng một thời điểm. Nếu không xử lý tốt, hệ thống sẽ gặp hiện tượng mất đồng bộ trạng thái hoặc nghiêm trọng hơn là tạo ra lỗ hổng bảo mật. Do đó, việc thiết kế một kiến trúc máy chủ backend có khả năng mở rộng tốt, chịu tải cao và tối ưu hóa băng thông mạng là nhu cầu thiết yếu để mang lại dịch vụ ổn định cho người dùng.

1.2 Mục tiêu và phạm vi đề tài

Trên thị trường hiện nay, có nhiều nền tảng máy chủ game thương mại sẵn có như Photon Engine hay Microsoft PlayFab. Mặc dù các dịch vụ này giúp đẩy nhanh tiến độ phát triển ban đầu, nhưng chúng thường có chi phí vận hành lớn và hạn chế khả năng tùy biến sâu của nhà phát triển. Ở hướng tiếp cận khác, việc tự xây dựng máy chủ đơn khối bằng Node.js hoặc Python tuy đơn giản nhưng lại gặp rào cản lớn về mặt hiệu năng xử lý đa luồng dưới tải cao do hạn chế về kiến trúc của các ngôn ngữ này.

Để giải quyết các hạn chế trên, đề án đặt mục tiêu thiết kế và phát triển hệ thống máy chủ backend theo kiến trúc vi dịch vụ (microservices) tối ưu hiệu năng cho

trò chơi thẻ tướng chiến thuật "The First Sect Origin". Phạm vi thực hiện của đề tài bao gồm thiết kế các dịch vụ độc lập là dịch vụ đăng nhập (Login), cổng kết nối (Gateway), quản lý thế giới game (World) và mô phỏng trận đấu (Combat) theo mô hình vi dịch vụ [1]; xây dựng ứng dụng client trên Unity [2]; đồng thời tích hợp hệ thống bảng xếp hạng thời gian thực. Đề án tiến hành thử nghiệm hiệu năng và khả năng chịu tải của hệ thống backend tự phát triển nhằm chứng minh tính khả thi của giải pháp.

1.3 Định hướng giải pháp

Để hiện thực hóa mục tiêu đã đặt ra, đề án lựa chọn phát triển hệ thống backend theo mô hình vi dịch vụ sử dụng ngôn ngữ lập trình Go và giao thức truyền thông gRPC. Ngôn ngữ Go [3] được chọn vì hiệu năng thực thi cao và cơ chế xử lý đồng thời siêu nhẹ Goroutine giúp máy chủ phục vụ lượng lớn kết nối đồng thời với tài nguyên tối thiểu. Giao thức gRPC [4] kết hợp định dạng Protocol Buffers được sử dụng thay thế cho chuẩn REST/JSON để giảm thiểu dung lượng dữ liệu trên đường truyền và tăng tốc độ tuần tự hóa gói tin. Hệ thống sử dụng PostgreSQL [5] để lưu trữ dữ liệu bền vững và tích hợp Redis [6] để quản lý phiên đăng nhập.

Đóng góp chính của đề án là thiết kế và triển khai thành công một kiến trúc backend vi dịch vụ hoàn chỉnh cho game trực tuyến, giải quyết bài toán điều phối yêu cầu giữa các dịch vụ và xây dựng được bảng xếp hạng thời gian thực hoạt động hiệu quả mà không làm nghẽn cơ sở dữ liệu PostgreSQL.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày kết quả khảo sát các hệ thống tương tự, từ đó phân tích các yêu cầu chức năng và yêu cầu phi chức năng của hệ thống trò chơi.

Chương 3 giới thiệu các nền tảng lý thuyết và công nghệ cốt lõi được sử dụng trong đề án như ngôn ngữ Go, gRPC, Protocol Buffers, PostgreSQL và Redis.

Chương 4 tập trung phân tích thiết kế chi tiết kiến trúc hệ thống, mô hình cơ sở dữ liệu, các luồng tương tác giữa các dịch vụ, và thuật toán xử lý logic trò chơi. Phần cuối chương trình bày kết quả thử nghiệm hiệu năng của hệ thống.

Chương 5 mô tả các giải pháp kỹ thuật tối ưu hóa hiệu năng hệ thống, bao gồm cải tiến luồng truyền dữ liệu qua gRPC, kiểm soát xử lý đồng thời bằng Goroutine và tối ưu hóa truy vấn cập nhật bảng xếp hạng thời gian thực.

Chương 6 tổng kết các kết quả đạt được, chỉ ra hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo của đề án.

Chương này đã giới thiệu khái quát về bối cảnh đề tài, sự cấp thiết của việc xây dựng backend game trực tuyến hiệu năng cao và định hướng giải pháp kỹ thuật. Những phân tích chi tiết về yêu cầu hệ thống sẽ được trình bày cụ thể trong Chương 2.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 2 này trình bày quá trình khảo sát hiện trạng và phân tích yêu cầu đối với hệ thống trò chơi trực tuyến "The First Sect Origin". Nội dung chương gồm các phần chính: khảo sát các hệ thống tương tự và đề xuất tính năng ở phần 2.1; xây dựng biểu đồ use case tổng quát và phân rã các chức năng cốt lõi ở phần 2.2; đặc tả chi tiết các use case quan trọng ở phần 2.3; và xác định các yêu cầu phi chức năng cùng các ràng buộc kỹ thuật ở phần 2.4.

2.1 Khảo sát hiện trạng

Trong quá trình phát triển các trò chơi trực tuyến nhiều người chơi, việc thiết kế hệ thống backend có hiệu năng cao và ổn định là yếu tố quyết định sự thành bại của sản phẩm. Để có cơ sở thiết kế hệ thống, đồ án tiến hành khảo sát và so sánh ba hướng tiếp cận phát triển backend phổ biến hiện nay:

1. Sử dụng máy chủ đơn khối (Monolithic Server) với Node.js hoặc Python:

Đây là giải pháp phổ biến cho các dự án nhỏ nhờ sự đơn giản trong triển khai và tài liệu phong phú. Tuy nhiên, kiến trúc đơn khối rất khó mở rộng quy mô khi số lượng người chơi tăng đột biến. Đồng thời, cơ chế xử lý Single-thread của Node.js hoặc Global Interpreter Lock (GIL) của Python gây ra nghẽn cổ chai nghiêm trọng đối với các tác vụ tính toán logic nặng như mô phỏng trận đấu thời gian thực hoặc sắp xếp bảng xếp hạng.

2. Sử dụng các dịch vụ Cloud Backend thương mại (Photon Engine, PlayFab):

Hướng tiếp cận này giúp rút ngắn đáng kể thời gian phát triển ban đầu thông qua các API tích hợp sẵn. Tuy nhiên, nhà phát triển phải đối mặt với chi phí vận hành tăng lũy tiến cực kỳ tốn kém theo số lượng người dùng. Bên cạnh đó, các dịch vụ này là hệ thống mã nguồn đóng, gây hạn chế lớn trong việc tùy biến sâu các thuật toán logic game đặc thù.

3. Tự phát triển hệ thống dựa trên kiến trúc vi dịch vụ (Microservices) với Go:

Giải pháp này chia nhỏ hệ thống máy chủ thành các dịch vụ độc lập kết nối với nhau qua gRPC. Sử dụng ngôn ngữ Go mang lại hiệu năng thực thi tương đương mã máy và khả năng xử lý song song vượt trội qua Goroutine. Dù kiến trúc này đòi hỏi thời gian thiết kế hệ thống ban đầu phức tạp hơn, nhưng nó mang lại khả năng mở rộng linh hoạt và tối ưu chi phí vận hành.

Dựa trên kết quả khảo sát và đối chiếu với mục tiêu xây dựng một trò chơi thể tướng chiến thuật trực tuyến có khả năng chịu tải tốt và dễ dàng mở rộng, đồ án lựa chọn hướng tiếp cận thứ ba: tự phát triển hệ thống backend theo kiến trúc vi

dịch vụ sử dụng Go. Các tính năng cốt lõi cần phát triển bao gồm hệ thống đăng ký/dăng nhập xác thực bảo mật, quản lý tài nguyên và xây dựng tông môn, chiêu mộ và huấn luyện đệ tử, mô phỏng trận đấu chiến thuật tự động, cùng bảng xếp hạng thời gian thực.

2.2 Tổng quan chức năng

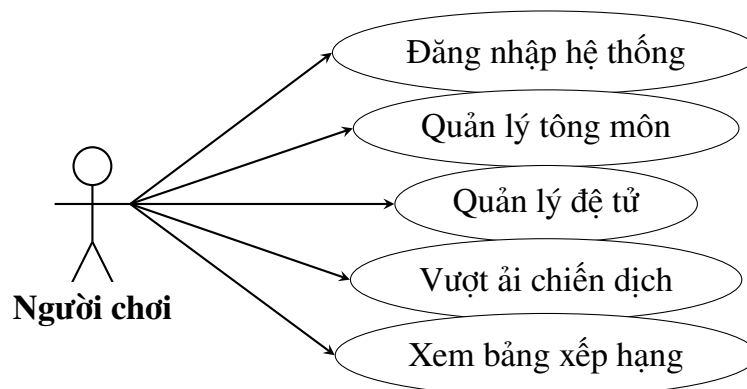
Hệ thống trò chơi "The First Sect Origin" bao gồm các nhóm chức năng chính được phân rõ để đảm bảo tính module và hiệu năng vận hành.

2.2.1 Biểu đồ use case tổng quát

Hệ thống tương tác chính giữa hai tác nhân (Actor):

- **Người chơi (Player):** Tác nhân chính tham gia vào trò chơi, thực hiện các hành động đăng nhập, xây dựng tông môn, quản lý đệ tử, vượt ải và xem bảng xếp hạng.
- **Hệ thống (System):** Thực hiện các tác vụ tự động như kiểm tra điều kiện xây dựng, tính toán kết quả trận đấu và cập nhật bảng xếp hạng.

Biểu đồ use case tổng quát bao gồm các nhóm chức năng chính: Đăng nhập hệ thống, Quản lý tông môn, Quản lý đệ tử, Vượt ải chiến dịch, và Xem bảng xếp hạng.



Hình 2.1: Biểu đồ use case tổng quát của hệ thống

2.2.2 Biểu đồ use case phân rõ các phân hệ chính

Dưới đây là biểu đồ phân rõ của các nhóm chức năng cốt lõi:

1. **Phân hệ xác thực:** Gồm các use case đăng ký tài khoản, đăng nhập hệ thống, xác thực qua mã thông báo JWT [7] và tự động điều phối phân vùng máy chủ khi quá tải (Roll Server).
2. **Phân hệ quản lý tông môn:** Gồm các use case xây dựng công trình mới, nâng cấp công trình, thu thập tài nguyên sản sinh từ công trình.

3. **Phân hệ quản lý đệ tử:** Gồm các use case chiêu mộ đệ tử ngẫu nhiên, nâng cấp đệ tử, và sắp xếp đội hình chiến đấu.
4. **Phân hệ vượt ải:** Gồm các use case chọn ải chiến dịch, gửi đội hình xuất trận, mô phỏng tính toán trận đấu, và nhận phần thưởng vượt ải.

2.2.3 Quy trình nghiệp vụ cốt lõi

Quy trình nghiệp vụ quan trọng nhất của hệ thống là luồng vận hành từ khi người chơi đăng nhập, tiến hành khai thác tài nguyên tại tông môn để nâng cấp đệ tử, sau đó tham gia vượt ải chiến dịch để tích lũy điểm lực chiến và cập nhật thứ hạng trên bảng xếp hạng thời gian thực. Quy trình này đòi hỏi sự phối hợp đồng bộ giữa dịch vụ Gateway nhận yêu cầu, dịch vụ World quản lý tài sản người chơi và dịch vụ Combat thực hiện tính toán kết quả trận đấu một cách độc lập.

2.3 Đặc tả chức năng

Dưới đây là đặc tả chi tiết của 5 use case quan trọng nhất trong hệ thống trò chơi.

2.3.1 Đặc tả use case Đăng nhập hệ thống

- **Tên use case:** Đăng nhập hệ thống. **Tác nhân:** Người chơi.
- **Tiền điều kiện:** Người chơi đã tải ứng dụng Client và có tài khoản đã đăng ký trên hệ thống.
- **Hậu điều kiện:** Người chơi nhận được mã xác thực JWT, thiết lập kết nối ổn định tới dịch vụ Gateway để bắt đầu phiên chơi.
- **Luồng sự kiện chính:**
 1. Người chơi nhập thông tin tài khoản (username/password) trên giao diện đăng nhập.
 2. Client gửi yêu cầu đăng nhập đến dịch vụ Login.
 3. Dịch vụ Login kiểm tra thông tin tài khoản trong cơ sở dữ liệu PostgreSQL.
 4. Hệ thống xác thực thành công, tạo mã JWT chứa thông tin người chơi, đồng thời lưu phiên làm việc vào Redis.
 5. Dịch vụ Login gửi trả mã JWT về phía Client.
 6. Client sử dụng mã JWT này để kết nối và xác thực với dịch vụ Gateway.
- **Luồng sự kiện phát sinh:**
 - *Sai mật khẩu:* Hệ thống thông báo lỗi đăng nhập và yêu cầu người chơi

kiểm tra lại thông tin.

- *Tài khoản chưa đăng ký*: Hệ thống gợi ý người chơi chuyển qua giao diện đăng ký tài khoản mới.

2.3.2 Đặc tả use case Xây dựng và nâng cấp công trình

- **Tên use case**: Xây dựng và nâng cấp công trình.
- **Tác nhân**: Người chơi.
- **Tiền điều kiện**: Người chơi đã đăng nhập, sở hữu đủ tài nguyên (Gỗ, Đá, Linh thạch) và đạt yêu cầu cấp độ tổng môn tối thiểu.
- **Hậu điều kiện**: Trạng thái công trình trong cơ sở dữ liệu được cập nhật, lượng tài nguyên tương ứng của người chơi bị khấu trừ.
- **Luồng sự kiện chính**:
 1. Người chơi chọn một khu đất trống hoặc chọn một công trình hiện có để xây dựng hoặc nâng cấp.
 2. Client gửi yêu cầu kèm ID công trình và loại hành động qua Gateway đến World-server.
 3. World-server kiểm tra số dư tài nguyên và các điều kiện tiên quyết trong cơ sở dữ liệu.
 4. Hệ thống thực hiện trừ tài nguyên của người chơi và cập nhật cấp độ mới của công trình trong PostgreSQL.
 5. World-server gửi phản hồi thành công kèm trạng thái tài nguyên mới về Client.
 6. Client cập nhật giao diện tổng môn của người chơi.
- **Luồng sự kiện phát sinh**:
 - *Không đủ tài nguyên*: Hệ thống từ chối yêu cầu, gửi mã lỗi về Client để hiển thị cảnh báo cho người chơi.

2.3.3 Đặc tả use case Chiêu mộ đệ tử

- **Tên use case**: Chiêu mộ đệ tử.
- **Tác nhân**: Người chơi.
- **Tiền điều kiện**: Người chơi sở hữu đủ vật phẩm hoặc đơn vị tiền tệ dùng để chiêu mộ đệ tử.
- **Hậu điều kiện**: Một thẻ đệ tử mới được khởi tạo thành công và lưu vào danh sách sở hữu của người chơi.

• **Luồng sự kiện chính:**

1. Người chơi truy cập giao diện chiêu mộ và nhấn chọn lượt chiêu mộ.
2. Client gửi yêu cầu chiêu mộ đệ tử qua dịch vụ Gateway tới World-server.
3. World-server thực hiện khâu trừ vật phẩm chiêu mộ của người chơi.
4. Hệ thống áp dụng thuật toán quay số ngẫu nhiên dựa trên bảng tỷ lệ xuất hiện (rarity weights) để chọn đệ tử.
5. World-server lưu thông tin đệ tử mới vào PostgreSQL dưới dạng thẻ bài sở hữu của người chơi.
6. World-server trả về thông tin chi tiết đệ tử nhận được cho Client để hiển thị hiệu ứng.

2.3.4 Đặc tả use case Vượt ải chiến dịch

- **Tên use case:** Vượt ải chiến dịch.
- **Tác nhân:** Người chơi.
- **Tiền điều kiện:** Người chơi đã thiết lập đội hình đệ tử xuất trận và chọn ải chiến dịch muốn tham gia.
- **Hậu điều kiện:** Kết quả trận đấu được tính toán, cập nhật tiến trình vượt ải và phân phối phần thưởng.

• **Luồng sự kiện chính:**

1. Người chơi chọn ải chiến dịch và nhấn nút bắt đầu trận đấu.
2. Client gửi yêu cầu vượt ải kèm cấu hình đội hình hiện tại qua Gateway đến Combat-server.
3. Combat-server lấy thông tin chỉ số đệ tử và quái vật của ải đấu để thực hiện mô phỏng trận đấu tự động theo lượt.
4. Combat-server tạo ra log trận đấu và xác định kết quả thắng/thua gửi về cho World-server.
5. Nếu người chơi thắng, World-server cập nhật tiến trình vượt ải mới nhất và cộng phần thưởng (Linh thạch, vật phẩm) vào tài khoản người chơi trong PostgreSQL.
6. Hệ thống trả kết quả và log mô phỏng về Client để tái hiện trận đấu dưới dạng hoạt ảnh trực quan cho người chơi.

2.3.5 Đặc tả use case Xem bảng xếp hạng thời gian thực

- **Tên use case:** Xem bảng xếp hạng thời gian thực.

- **Tác nhân:** Người chơi.
- **Tiền điều kiện:** Người chơi bấm vào biểu tượng bảng xếp hạng trên giao diện sảnh chính.
- **Hậu điều kiện:** Danh sách người chơi có thứ hạng cao nhất hiển thị đầy đủ trên màn hình Client.
- **Luồng sự kiện chính:**
 1. Người chơi gửi yêu cầu xem bảng xếp hạng từ Client.
 2. Yêu cầu được gửi qua Gateway tới World-server.
 3. World-server truy vấn danh sách người chơi xếp hạng theo điểm lực chiến hoặc tiến trình vượt ải từ cơ sở dữ liệu.
 4. World-server gửi trả danh sách xếp hạng đã định dạng về Client.
 5. Client hiển thị danh sách bảng xếp hạng lên màn hình người chơi.

2.3.6 Đặc tả use case Tự động điều phối vùng máy chủ khi quá tải (Roll Server)

- **Tên use case:** Tự động điều phối vùng máy chủ khi quá tải (Roll Server).
- **Tác nhân:** Hệ thống, Người chơi.
- **Tiền điều kiện:** Người chơi gửi yêu cầu đăng nhập vào game. Phân vùng máy chủ hiện tại đã đạt ngưỡng chịu tải cấu hình tối đa.
- **Hậu điều kiện:** Hệ thống tự động ghi nhận phân vùng máy chủ mới (ví dụ: chuyển từ *zone1* sang *zone2*) và điều hướng kết nối của người chơi đến máy chủ mới.
- **Luồng sự kiện chính:**
 1. Người chơi gửi yêu cầu đăng nhập từ Client đến dịch vụ Gateway.
 2. Dịch vụ Gateway/Login kiểm tra số lượng người chơi hoạt động đồng thời (CCU) trên phân vùng máy chủ hiện tại (ví dụ: *zone1*).
 3. Hệ thống phát hiện số lượng CCU vượt quá ngưỡng chịu tải tối đa của máy chủ hiện tại.
 4. Hệ thống kích hoạt cơ chế Roll Server, tự động cấp mã thông báo JWT mới chứa thông tin phân vùng máy chủ mới (ví dụ: *zone2*).
 5. Client nhận phản hồi đăng nhập chứa JWT mới và tự động thiết lập kết nối đến phân vùng mới.

6. Nhân vật mới của người chơi được khởi tạo biệt lập trên phân vùng mới thông qua khóa duy nhất (*user_id*, *server_id*) trong cơ sở dữ liệu PostgreSQL.

- **Luồng sự kiện phát sinh:**

- *Phân vùng hiện tại chưa quá tải*: Hệ thống tiếp tục giữ phiên kết nối và dữ liệu của người chơi tại phân vùng hiện tại.

2.4 Yêu cầu phi chức năng

Để đảm bảo hệ thống vận hành tốt dưới tải lớn và mang lại trải nghiệm mượt mà, các yêu cầu phi chức năng sau đây được thiết lập:

- **Hiệu năng và tải (Performance & Scalability):**

- Hệ thống backend được thiết kế để đáp ứng khả năng xử lý đồng thời lượng lớn người chơi ảo (simulated users) trong các kịch bản kiểm thử tải.
 - Thời gian phản hồi trung bình (latency) của dịch vụ Gateway đối với các yêu cầu thông thường của người chơi phải nhỏ hơn 200ms dưới điều kiện mạng ổn định.
 - Dịch vụ mô phỏng trận đấu (Combat-server) phải hoàn thành tính toán một trận đấu trong thời gian dưới 50ms để đảm bảo không xảy ra nghẽn hàng đợi xử lý.

- **Độ tin cậy và Nhất quán dữ liệu (Reliability & Consistency):**

- Hệ thống hoạt động liên tục với độ sẵn sàng dịch vụ tối thiểu là 99,5%.
 - Đảm bảo tính nhất quán giao dịch tuyệt đối đối với các hành động khấu trừ tài nguyên (Gỗ, Đá, Linh thạch) và trao thưởng vật phẩm, loại bỏ hoàn toàn khả năng xảy ra lỗi trùng lặp giao dịch (double-spending).

- **Bảo mật và An toàn thông tin (Security):**

- Mọi kết nối truyền tải thông tin giữa Client và Gateway phải được bảo vệ qua các kênh truyền bảo mật (gRPC over TLS).
 - Quản lý phiên kết nối chặt chẽ bằng cơ chế kiểm tra tính hợp lệ của JWT đã lưu trong Redis, ngăn chặn các hình thức tấn công giả mạo yêu cầu.

- **Ràng buộc kỹ thuật (Technical Constraints):**

- Hệ thống backend được phát triển hoàn toàn bằng ngôn ngữ Go (phiên bản 1.20 trở lên).
 - Hệ quản trị cơ sở dữ liệu sử dụng PostgreSQL 15 để lưu trữ dữ liệu bền

vững và Redis 7 làm bộ nhớ đệm tốc độ cao.

- Giao diện và logic client được xây dựng trên công cụ Unity 6.

Chương này đã làm rõ các yêu cầu chức năng và phi chức năng của hệ thống qua việc khảo sát hiện trạng, xây dựng biểu đồ use case và đặc tả chi tiết các nghiệp vụ chính. Đây là cơ sở quan trọng để tiến hành phân tích thiết kế kiến trúc hệ thống và lựa chọn công nghệ triển khai phù hợp trong chương tiếp theo – Chương 3.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày chi tiết về các nền tảng lý thuyết và công nghệ cốt lõi được sử dụng để thiết kế và xây dựng hệ thống trò chơi trực tuyến nhiều người chơi "The First Sect Origin". Các công nghệ được lựa chọn bao gồm: ngôn ngữ lập trình Go cho hệ thống máy chủ, giao thức truyền thông gRPC kết hợp Protocol Buffers, hệ quản trị cơ sở dữ liệu PostgreSQL, hệ thống lưu trữ dữ liệu Redis, và game engine Unity 6 cho phía client. Báo cáo tiến hành phân tích lý do lựa chọn từng công nghệ, các giải pháp thay thế tương ứng, và cách thức chúng giải quyết các yêu cầu thực tế đã đặt ra ở Chương 2.

3.1 Ngôn ngữ lập trình Go cho hệ thống máy chủ

Hệ thống máy chủ backend của trò chơi đòi hỏi khả năng xử lý đồng thời cao (concurrency) để phục vụ hàng ngàn người chơi tương tác cùng lúc, đồng thời cần tối ưu hóa tài nguyên phần cứng của máy chủ (CPU, RAM). Để đáp ứng yêu cầu này, ngôn ngữ Go [3] được lựa chọn làm ngôn ngữ phát triển chính cho các dịch vụ Gateway, Login, World và Combat.

3.1.1 Đặc điểm nổi bật

Go là ngôn ngữ biên dịch trực tiếp ra mã máy (compiled language), giúp đạt tốc độ thực thi tiệm cận C/C++ mà không gặp độ trễ lớn như các ngôn ngữ thông dịch hoặc chạy trên máy ảo (như Java, C#). Đặc biệt, Go cung cấp cơ chế xử lý đồng thời siêu nhẹ dựa trên *Goroutines* và kết nối giữa các luồng thông qua *Channels*:

- **Goroutine**: Là luồng ảo được quản lý bởi bộ lập lịch của Go (Go Runtime Scheduler) chứ không phải luồng hệ điều hành (OS Thread). Mỗi Goroutine chỉ tiêu tốn khoảng 2 KB bộ nhớ khi khởi tạo (so với 1 MB đến 8 MB của OS Thread). Điều này cho phép một máy chủ có cấu hình RAM trung bình (ví dụ: 8 GB RAM) chạy đồng thời hàng triệu Goroutine (tiêu tốn khoảng 2 GB RAM cho không gian ngăn xếp của luồng) mà không làm cạn kiệt tài nguyên RAM của hệ thống.
- **Scheduler (M:N)**: Go Runtime phân phối hệ thống M Goroutines lên N luồng hệ điều hành một cách tự động và tối ưu, giúp tận dụng tối đa sức mạnh của bộ vi xử lý đa nhân.

3.1.2 Giải pháp thay thế và Lý do lựa chọn

Để phát triển máy chủ web/game, các lựa chọn thay thế phổ biến bao gồm Node.js (JavaScript) và Python:

Tiêu chí so sánh	Node.js (JavaScript)	Python	Go (Lựa chọn)
Mô hình xử lý	Đơn luồng với Event Loop (Asynchronous I/O).	Đơn luồng bị giới hạn bởi cơ chế khóa GIL.	Đa luồng thực sự thông qua cơ chế Goroutine.
Hiệu năng CPU-bound	Yếu, nghẽn luồng chính khi chạy tính toán nặng.	Thấp, tốc độ thông dịch chậm.	Rất cao, biên dịch trực tiếp ra mã máy thuần túy.
Mức tiêu thụ tài nguyên	Trung bình (chạy trên V8 engine).	Cao (tốn RAM và CPU tải thông dịch).	Cực kỳ thấp (tối ưu hóa bộ nhớ tĩnh).

Bảng 3.1: Bảng so sánh lựa chọn công nghệ phát triển máy chủ backend

- **Node.js:** Mặc dù xử lý các tác vụ I/O bất đồng bộ rất tốt, Node.js hoạt động trên mô hình đơn luồng. Khi máy chủ Combat phải tính toán liên tiếp kết quả mô phỏng của hàng loạt trận đấu (tác vụ nặng về tính toán CPU-bound), luồng chính của Node.js sẽ bị nghẽn, dẫn đến việc tất cả các yêu cầu khác của người chơi bị trì trệ.
- **Python:** Gặp hạn chế nghiêm trọng về hiệu năng thực thi và cơ chế Global Interpreter Lock (GIL) ngăn cản việc chạy đa luồng thực thụ để tối ưu hóa CPU đa nhân.
- **Lý do lựa chọn Go:** Go kết hợp được cả hai ưu điểm: hiệu năng I/O bất đồng bộ cực cao như Node.js nhờ cơ chế Network Poller ngầm, và hiệu năng tính toán CPU-bound mạnh mẽ nhờ biên dịch trực tiếp ra mã máy và hỗ trợ đa luồng thực sự. Điều này giúp giải quyết triệt để yêu cầu phi chức năng về hiệu năng và độ ổn định của hệ thống máy chủ dưới tải cao.

3.2 Giao thức truyền thông gRPC và Protocol Buffers

Trong kiến trúc vi dịch vụ (microservices) được áp dụng cho trò chơi [1], các dịch vụ cần giao tiếp với nhau liên tục để đồng bộ hóa trạng thái tài sản người chơi và kết quả trận đấu. Ngoài ra, Client game trên Unity cũng cần kết nối với máy chủ backend với độ trễ tối thiểu. Giao thức gRPC [4] kết hợp định dạng Protocol Buffers được lựa chọn làm giải pháp truyền thông chính.

3.2.1 Đặc điểm nổi bật

gRPC là RPC framework mã nguồn mở chạy trên nền tảng giao thức truyền tải HTTP/2. Điểm đặc trưng của gRPC là sử dụng Protocol Buffers (Protobuf) làm ngôn ngữ định nghĩa giao diện (IDL) và định dạng tuần tự hóa dữ liệu:

- **HTTP/2:** Hỗ trợ cơ chế ghép kênh (Multiplexing) cho phép gửi nhiều yêu cầu

và nhận nhiều phản hồi đồng thời trên cùng một kết nối TCP duy nhất, loại bỏ việc phải mở mới kết nối liên tục (như HTTP/1.1), giúp giảm đáng kể độ trễ kết nối.

- **Tuần tự hóa nhị phân:** Protobuf mã hóa dữ liệu thành định dạng nhị phân siêu nhỏ gọn trước khi truyền trên đường truyền, thay vì dùng dạng văn bản như JSON hay XML.

3.2.2 Giải pháp thay thế và Lý do lựa chọn

Giải pháp truyền thống phổ biến nhất là xây dựng các API theo chuẩn REST sử dụng định dạng JSON (REST/JSON):

- **Hạn chế của REST/JSON:** JSON là định dạng văn bản (text-based), chứa nhiều ký tự thừa (ngoặc nhọn, dấu nháy, tên thuộc tính lặp đi lặp lại) làm tăng dung lượng gói tin. Quá trình chuyển đổi cấu trúc dữ liệu trong bộ nhớ sang chuỗi JSON (serialization) và ngược lại (deserialization) tiêu tốn rất nhiều chu kỳ xử lý của CPU. Ngoài ra, REST chạy trên HTTP/1.1 gặp hiện tượng nghẽn đầu đường truyền (Head-of-Line Blocking).
- **Lý do lựa chọn gRPC:** Sử dụng truyền tải nhị phân giúp dung lượng gói tin giảm từ 60% đến 80% so với JSON tương đương, tốc độ tuần tự hóa nhanh gấp 5 đến 10 lần. Cơ chế HTTP/2 Multiplexing giúp Client duy trì một kết nối duy nhất đến Gateway-server, truyền tải gói tin nhanh chóng để cập nhật trạng thái game thời gian thực mà không làm tăng tải của card mạng.

3.3 Hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL

Cơ sở dữ liệu của trò chơi trực tuyến lưu trữ các thông tin cốt lõi về tài khoản, tài sản người chơi (Gỗ, Đá, Linh thạch), thông tin đệ tử và cấp độ công trình tông môn. Hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL [5] được lựa chọn làm kho lưu trữ dữ liệu bền vững.

3.3.1 Đặc điểm nổi bật

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ, tuân thủ nghiêm ngặt các tính chất ACID (Atomicity, Consistency, Isolation, Durability) của giao dịch (transaction). PostgreSQL hỗ trợ các cơ chế khóa (locking) và mức độ cô lập giao dịch nâng cao (như Serializable), cùng khả năng mở rộng tốt thông qua phân vùng dữ liệu (partitioning).

3.3.2 Giải pháp thay thế và Lý do lựa chọn

Một lựa chọn thay thế rất phổ biến trong phát triển game là các hệ cơ sở dữ liệu NoSQL như MongoDB:

- **MongoDB:** Lưu trữ dữ liệu dạng tài liệu (JSON document) linh hoạt, dễ thay đổi cấu trúc mà không cần khai báo schema trước. Tuy nhiên, MongoDB thiếu khả năng ràng buộc toàn vẹn dữ liệu mạnh mẽ và các giao dịch liên quan đến nhiều tài liệu (multi-document transactions) rất phức tạp và hiệu năng kém hơn so với cơ sở dữ liệu quan hệ truyền thống.
- **Lý do lựa chọn PostgreSQL:** Trong game chiến thuật thẻ tướng, các giao dịch tài sản yêu cầu tính chính xác và nhất quán tuyệt đối. Ví dụ, khi người chơi thực hiện nâng cấp một công trình tông môn: hệ thống phải kiểm tra số dư tài nguyên, trừ lượng tài nguyên tương ứng (Gỗ, Đá, Linh thạch) và tăng cấp độ công trình trong cơ sở dữ liệu. Tất cả các bước này phải nằm trong một Transaction duy nhất. Nếu bất kỳ bước nào lỗi (ví dụ mất kết nối mạng hoặc lỗi server giữa chừng), toàn bộ giao dịch phải được rollback để tránh trường hợp tài nguyên bị trừ nhưng công trình không tăng cấp, hoặc ngược lại. PostgreSQL cung cấp độ tin cậy giao dịch ACID tối đa để giải quyết bài toán này.

3.4 Hệ thống lưu trữ dữ liệu Redis

Hệ thống yêu cầu cập nhật và truy xuất bảng xếp hạng lực chiến thời gian thực của toàn bộ người chơi. Nếu thực hiện truy vấn này trực tiếp trên cơ sở dữ liệu đĩa như PostgreSQL sẽ gây ra quá tải nghiêm trọng. Hệ thống bộ nhớ đệm Redis [6] được tích hợp để xử lý bài toán này.

3.4.1 Đặc điểm nổi bật

Redis là hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ RAM (in-memory data store) hoạt động dưới dạng key-value với hiệu năng cực cao (tốc độ đọc ghi hơn 100,000 requests/giây với độ trễ dưới 1 mili-giây). Redis hỗ trợ các cấu trúc dữ liệu nâng cao như String, Hash, List, Set và đặc biệt là Sorted Set.

3.4.2 Giải pháp thay thế và Lý do lựa chọn

Giải pháp thay thế là lưu trữ điểm lực chiến trong PostgreSQL và thực hiện truy vấn sắp xếp trực tiếp khi người chơi yêu cầu xem bảng xếp hạng:

- **Hạn chế của việc truy vấn trực tiếp SQL:** Mỗi khi người chơi mở giao diện bảng xếp hạng, hệ thống phải thực hiện lệnh `SELECT ... ORDER BY luc_chien DESC LIMIT 100`. Khi số lượng người chơi lớn (hàng chục ngàn người chơi hoạt động đồng thời), việc quét bảng dữ liệu liên tục để sắp xếp lại sẽ vắt kiệt băng thông I/O đĩa cứng của PostgreSQL, gây treo toàn bộ hệ thống cơ sở dữ liệu.
- **Lý do lựa chọn Redis:** Redis cung cấp cấu trúc dữ liệu *Sorted Set* (ZSET).

Trong ZSET, mỗi phần tử (ID người chơi) được liên kết với một điểm số (lực chiến). Redis tự động sắp xếp các phần tử này trong bộ nhớ bằng cấu trúc dữ liệu Skip List kết hợp với Hash Table. Việc cập nhật điểm lực chiến mới của người chơi chỉ mất độ phức tạp thuật toán là $O(\log(N))$, và việc lấy danh sách top 100 người chơi hàng đầu mất độ phức tạp $O(\log(N) + M)$ (với $M = 100$). Tác vụ này chạy hoàn toàn trên RAM với tốc độ tức thời, giúp giảm tải hoàn toàn cho PostgreSQL và đáp ứng yêu cầu hiển thị bảng xếp hạng thời gian thực.

3.5 Game Engine Unity 6 cho ứng dụng Client

Ứng dụng phía Client có nhiệm vụ hiển thị đồ họa 2D trực quan, tái hiện các trận đấu thẻ tướng sống động, và giao tiếp với hệ thống backend qua mạng. Game engine Unity 6 [2] được lựa chọn làm công cụ phát triển chính.

3.5.1 Đặc điểm nổi bật

Unity 6 (phiên bản mới nhất của dòng engine Unity) cải tiến mạnh mẽ về hiệu năng kết xuất đồ họa thông qua SRP (Scriptable Render Pipeline), giảm thiểu bộ nhớ tiêu thụ và tăng tốc độ tải tài nguyên. Unity sử dụng ngôn ngữ lập trình C# và hỗ trợ công nghệ IL2CPP để biên dịch mã nguồn C# trực tiếp thành mã nguồn C++ thuần túy trước khi đóng gói ra ứng dụng, giúp tối ưu hóa tối đa hiệu năng chạy trên thiết bị di động Android.

3.5.2 Giải pháp thay thế và Lý do lựa chọn

Các lựa chọn thay thế bao gồm Unreal Engine hoặc Godot:

- **Unreal Engine:** Quá nặng đối với thể loại game thẻ tướng chiến thuật 2D cấu hình nhẹ, dung lượng gói cài đặt (APK/IPA) đầu ra quá lớn và không tối ưu cho các thiết bị di động tầm trung.
- **Godot Engine:** Mặc dù nhẹ và mã nguồn mở, hệ sinh thái của Godot còn non trẻ, thiếu các thư viện plugin hỗ trợ kết nối mạng (như thư viện tương thích gRPC cho C#) và cộng đồng hỗ trợ chưa đủ mạnh cho các dự án game thương mại.
- **Lý do lựa chọn Unity 6:** Cung cấp sự cân bằng hoàn hảo giữa hiệu năng đồ họa mạnh mẽ, dung lượng đóng gói tối ưu, và sự hỗ trợ phong phú từ Asset Store. Đặc biệt, Unity tích hợp tốt với thư viện gRPC dành cho C# chạy trên nền .NET Standard, giúp đồng bộ hóa dữ liệu trực tiếp với backend vi dịch vụ viết bằng Go một cách dễ dàng và đồng bộ.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày chi tiết về quá trình phân tích thiết kế, xây dựng ứng dụng, kiểm thử và triển khai thực tế hệ thống trò chơi trực tuyến nhiều người chơi "The First Sect Origin". Nội dung bao gồm việc lựa chọn và đặc tả thiết kế kiến trúc phần mềm, mô tả cấu trúc các gói, thiết kế các lớp đối tượng chính, vẽ sơ đồ thực thể liên kết cơ sở dữ liệu, liệt kê chi tiết các công cụ và thư viện phát triển, trình bày các kịch bản kiểm thử chức năng và cuối cùng là phương án triển khai hệ thống máy chủ chịu tải cao.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Trong việc phát triển hệ thống trò chơi trực tuyến nhiều người chơi đòi hỏi khả năng phản hồi nhanh và chịu tải lớn, thiết kế kiến trúc đóng vai trò quyết định độ ổn định lâu dài của sản phẩm. Hệ thống "The First Sect Origin" được thiết kế dựa trên sự kết hợp giữa kiến trúc **Vi dịch vụ (Microservices)** ở mức độ hệ thống tổng quát và kiến trúc **Clean Architecture** ở mức độ cấu trúc mã nguồn bên trong từng dịch vụ.

a, Mô hình kiến trúc Vi dịch vụ (Microservices)

Hệ thống máy chủ backend được chia tách thành các dịch vụ độc lập, chuyên biệt hóa nhiệm vụ và giao tiếp với nhau thông qua giao thức gRPC hiệu năng cao:

- **Gateway Service:** Đóng vai trò là điểm tiếp nhận yêu cầu (Single Entry Point) duy nhất từ phía client. Dịch vụ này thực hiện chức năng xác thực mã thông báo JWT của người dùng, giới hạn tần suất yêu cầu (Rate Limiting) nhằm chống tấn công từ chối dịch vụ, và điều phối các cuộc gọi gRPC tới các vi dịch vụ tương ứng ở mạng nội bộ. Đồng thời, Gateway còn chứa logic "Roll Server" để điều hướng người chơi kết nối vào phân vùng (Zone) thích hợp khi hệ thống mở rộng.
- **Login Server:** Chịu trách nhiệm xử lý toàn bộ các yêu cầu liên quan đến tài khoản người dùng, bao gồm đăng ký, đăng nhập, liên kết tài khoản mạng xã hội (Google, Apple, Facebook) và cấp phát token xác thực. Dịch vụ này truy cập trực tiếp vào cơ sở dữ liệu toàn cục (Global Database) chứa thông tin tài khoản dùng chung giữa các Zone.
- **World Server:** Quản lý toàn bộ trạng thái thế giới trò chơi và dữ liệu phiêu lưu của người chơi thuộc một phân vùng cụ thể. Các logic cốt lõi như xây dựng

và nâng cấp tông môn, sản sinh tài nguyên (Gỗ, Đá, Linh thạch), quản lý đệ tử, sắp xếp đội hình chiến đấu, thực hiện gacha chiêu mộ và theo dõi độ kiên nhẫn (pity) đều diễn ra tại đây. World Server lưu trữ trạng thái bền vững trong Game Database của riêng phân vùng đó.

- **Combat Server:** Dịch vụ tính toán phi trạng thái (Stateless) chuyên xử lý và mô phỏng các trận đấu thẻ tướng chiến thuật 2D. Khi người chơi tham gia vượt ải, World Server sẽ gửi thông tin đội hình người chơi và đội hình quái vật sang Combat Server qua gRPC. Combat Server thực hiện mô phỏng toàn bộ tiến trình trận đấu theo lượt, tính toán sát thương, hiệu ứng và trả về kết quả thắng/thua cùng nhật ký trận đấu chi tiết (battle log).

b, Mô hình Clean Architecture cho từng dịch vụ

Mỗi dịch vụ được tổ chức mã nguồn theo 4 lớp cốt lõi của Clean Architecture nhằm giảm thiểu sự phụ thuộc trực tiếp vào framework hoặc công cụ lưu trữ dữ liệu, tăng khả năng viết kiểm thử đơn vị độc lập:

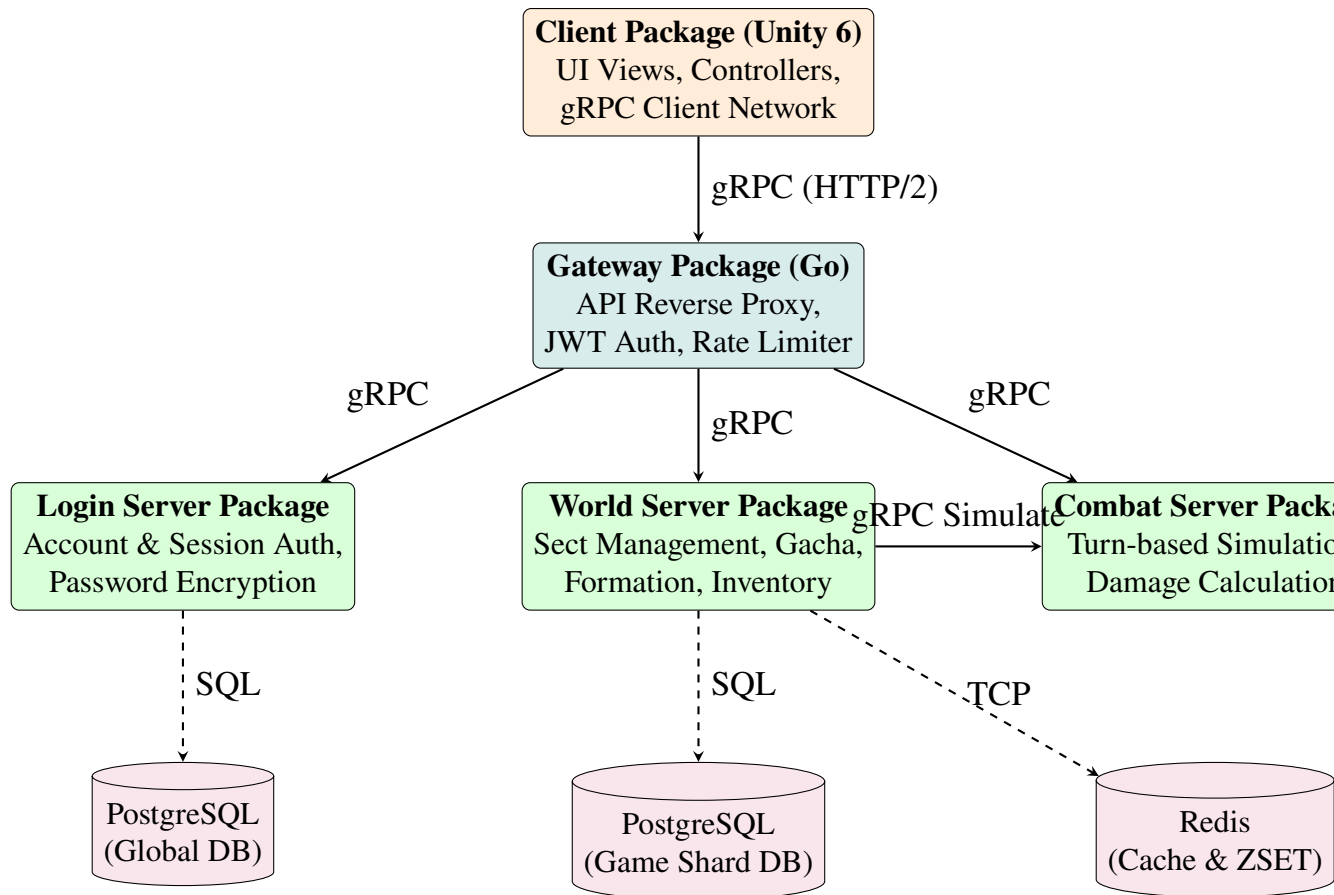
1. **Domain Entities Layer:** Chứa các định nghĩa cấu trúc dữ liệu thuần túy (như `Player`, `PlayerHero`, `PlayerBuilding`) đại diện cho thực thể nghiệp vụ. Lớp này không phụ thuộc vào bất kỳ thư viện ngoài hay hệ quản trị cơ sở dữ liệu nào.
2. **Use Cases / Services Layer:** Chứa toàn bộ quy tắc nghiệp vụ (Business Rules) của game. Ví dụ, dịch vụ `BuildingService` chứa logic kiểm tra điều kiện nâng cấp công trình (người chơi có đủ tài nguyên Gỗ, Đá, Linh thạch không, cấp độ của Điện chính là bao nhiêu) và thực hiện trừ tài nguyên trước khi bắt đầu nâng cấp.
3. **Interface Adapters (Handlers/Controllers) Layer:** Đóng vai trò cầu nối chuyển đổi dữ liệu. Trong dự án này, lớp này là các gRPC handlers nhận các đối tượng request dạng Protobuf (ví dụ `UpgradeBuildingRequest`), chuyển đổi sang dạng struct Go, gọi Use Case tương ứng và sau đó chuyển đổi kết quả thành Protobuf response trả về cho client.
4. **Infrastructure (Repositories/Drivers) Layer:** Chứa mã nguồn thao tác với cơ sở dữ liệu (PostgreSQL qua thư viện `pgx`, Redis qua `go-redis`) và giao tiếp ngoại vi. Lớp này hiện thực hóa các giao diện (Interface) được định nghĩa ở lớp nghiệp vụ thông qua cơ chế tiêm phụ thuộc (Dependency Injection).

4.1.2 Thiết kế tổng quan

Dưới đây trình bày sơ đồ gói UML (UML Package Diagram) thể hiện mối quan hệ phụ thuộc giữa các thành phần phần mềm trong toàn bộ hệ thống. Các gói được

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

phân chia thành các tầng rõ ràng theo quy tắc thiết kế kiến trúc: tầng trên phụ thuộc tầng dưới, các gói ở cùng một tầng nghiệp vụ không có sự phụ thuộc chéo lẫn nhau, và các giao tiếp mạng được chuẩn hóa qua gRPC.



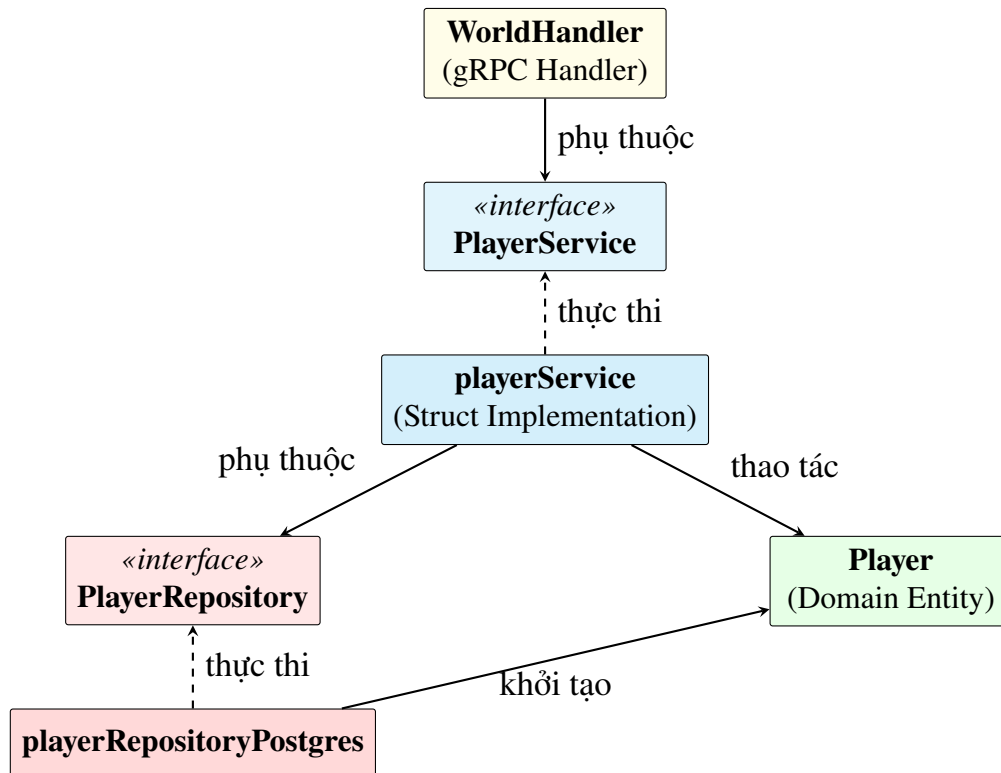
Hình 4.1: Biểu đồ gói UML thể hiện sự phụ thuộc và giao tiếp trong hệ thống

Giải thích nhiệm vụ chính của các gói:

- **Client Package (Unity 6):** Xử lý giao diện người dùng, tiếp nhận thao tác kéo thả và kết nối mạng đến Gateway qua thư viện gRPC.
- **Gateway Package (Go):** Là gói trung gian định tuyến. Nó không chứa cơ sở dữ liệu nghiệp vụ game mà chỉ xác thực mã thông báo và phân luồng yêu cầu.
- **Login Server Package:** Quản lý thông tin đăng nhập toàn cục, thực hiện giao dịch ghi nhận tài khoản mới vào PostgreSQL Global.
- **World Server Package:** Lưu trữ và cập nhật tiến trình của người chơi tại từng phân vùng, là gói nghiệp vụ phức tạp nhất liên kết với PostgreSQL Game và Redis.
- **Combat Server Package:** Thực hiện tính toán sát thương thuần túy, nhận thông tin chỉ số thẻ tướng từ World Server và trả về kết quả mô phỏng trận đấu.

4.1.3 Thiết kế chi tiết gói

Để giải quyết bài toán nghiệp vụ nâng cấp tông môn và thu hoạch tài nguyên trong World Server mà không bị phụ thuộc vào tầng dữ liệu cụ thể, hệ thống áp dụng nguyên lý đảo ngược phụ thuộc (Dependency Inversion). Biểu đồ thiết kế chi tiết gói dưới đây thể hiện các mối quan hệ lớp trong gói World.



Hình 4.2: Biểu đồ quan hệ giữa các thành phần trong gói World Server

Giải thích thiết kế mối quan hệ:

- **WorldHandler** phụ thuộc vào interface **PlayerService** để thực hiện các yêu cầu nghiệp vụ, giúp tách biệt giao thức truyền thông gRPC khỏi logic nghiệp vụ game.
- Struct **playerService** thực thi interface **PlayerService**, đóng gói logic thay đổi tài sản và công trình. Nó phụ thuộc vào interface dữ liệu **PlayerRepository**.
- Struct **playerRepositoryPostgres** nằm ở tầng cơ sở hạ tầng, thực thi interface **PlayerRepository** để thực hiện các câu lệnh SQL tác động vào bảng dữ liệu trong PostgreSQL.
- **Player** là thực thể nghiệp vụ chứa các thuộc tính và phương thức cơ bản về tài sản của người chơi. Cả dịch vụ và kho lưu trữ đều sử dụng lớp thực thể này để luân chuyển dữ liệu.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Thiết kế giao diện người dùng của "The First Sect Origin" hướng tới trải nghiệm mượt mà trên thiết bị di động Android. Đặc tả thông số thiết kế được thống nhất chung để đảm bảo tính nhất quán trên toàn hệ thống.

a, Đặc tả kỹ thuật giao diện

- **Độ phân giải đích:** Bản dựng cài đặt duy nhất của Client trên thiết bị di động Android (tệp APK) được tối ưu hóa hiển thị chạy trên màn hình độ phân giải thiết kế chuẩn là 2778×1284 (ngang), mật độ điểm ảnh 240 DPI. Để đảm bảo khả năng tương thích cao trên mọi loại màn hình khác, hệ thống UI trong Unity 6 sử dụng độ phân giải tham chiếu Canvas Scaler thống nhất là 1920×1080 kết hợp cơ chế tự động co giãn (Canvas Scaler - Match Width or Height) tùy thuộc vào tỷ lệ Aspect Ratio thực tế của thiết bị di động Android.
- **Số lượng màu sắc hỗ trợ:** Hệ màu RGB 32-bit (hơn 16,7 triệu màu) hỗ trợ hiển thị dải màu mượt mà, giúp các hoạt ảnh kỹ năng và hiệu ứng tông môn đạt hiệu quả thẩm mỹ cao nhất.

b, Chuẩn hóa thiết kế và bảng màu tông môn

Bảng màu thiết kế chủ đạo được xây dựng dựa trên chủ đề tu tiên hiệp cổ trang của trò chơi:

Thành phần	Mã màu Hex	Ý nghĩa trực quan	Cách sử dụng trên UI
Màu chủ đạo	#1F402B	Xanh lục đậm (Linh thạch)	Thanh tiêu đề, nền khung giao diện chính.
Màu nhấn mạnh	#D4AF37	Vàng hoàng kim (Đế vương)	Nút xác nhận chính, viền khung, sao đệ tử.
Màu tài nguyên	#4A2C11	Nâu gỗ cổ điển (Gỗ/Đá)	Nền thanh hiển thị tài nguyên, nút đóng cửa sổ.
Màu thông điệp	#FAF8F5	Trắng ngà thanh lịch	Màu chữ văn bản chính hiển thị trên giao diện.

Bảng 4.1: Bảng chuẩn hóa phối màu giao diện người dùng

Quy định chuẩn hóa các phần tử tương tác:

- **Thiết kế nút bấm (Button):** Nút bấm có góc bo tròn mặc định 8 pixel. Các nút hành động chính (như "Xác nhận", "Chiêu mộ") có màu nhấn vàng hoàng kim với viền nổi. Các nút hành động phụ (như "Hủy", "Quay lại") có màu nâu gỗ cổ điển. Khi người chơi di chuột qua (hover) hoặc bấm giữ (pressed), nút sẽ tự động thay đổi độ sáng 10% để tạo phản hồi thị giác (visual feedback).

• **Vị trí hiển thị thông điệp phản hồi (Feedback/Toast):**

- *Thông báo tạm thời (Toast Notification)*: Hiển thị ở vị trí phía trên chính giữa màn hình (Top-Center) trong vòng 2 giây rồi tự động mờ dần đi, áp dụng cho các thông điệp thu thập tài nguyên hoặc thông báo lỗi không nghiêm trọng.
- *Hộp thoại xác nhận (Modal Popup Dialog)*: Hiển thị ở chính giữa trung tâm màn hình (Screen Center), yêu cầu người chơi tương tác bấm nút để đóng lại, áp dụng cho các thông báo trúng tướng UR/SSR hoặc xác nhận mua sắm tài nguyên.

4.2.2 Thiết kế lớp

Mục này trình bày thiết kế chi tiết thuộc tính và phương thức của 3 lớp đối tượng quan trọng nhất quản lý trạng thái trò chơi tại máy chủ World Server: lớp người chơi (Player), lớp công trình của người chơi (PlayerBuilding) và lớp đệ tử (PlayerHero).

a, Chi tiết các lớp đối tượng cốt lõi

Lớp Player (Thực thể người chơi) Lớp này lưu giữ các chỉ số tài sản và thuộc tính cơ bản của một tài khoản tại một phân vùng game:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thuộc tính / Phương thức	Kiểu dữ liệu	Mục đích sử dụng
id	BIGINT (PK)	Định danh duy nhất người chơi trong Game DB.
user_id	BIGINT	Khóa ngoại liên kết sang tài khoản Global DB.
nickname	VARCHAR(32)	Biệt hiệu hiển thị trong trò chơi của người chơi.
level	INT	Cấp độ tổng môn hiện tại của người chơi.
exp	BIGINT	Điểm kinh nghiệm tích lũy để tăng cấp tổng môn.
gold	BIGINT	Lượng tài nguyên Gỗ tích lũy của người chơi.
diamond	BIGINT	Lượng tài nguyên Linh thạch quý hiếm tích lũy.
stamina	INT	Điểm thể lực để vượt ải chiến dịch.
last_stamina_at	TIMESTAMP	Thời điểm cập nhật và tự động hồi phục thể lực gần nhất.
AddResources (g, d)	void	Cộng thêm tài nguyên Gỗ (g) và Linh thạch (d).
DeductResources (g, d)	boolean	Trừ tài nguyên, trả về false nếu không đủ số dư.
AddExp (amount)	void	Cộng kinh nghiệm tổng môn và xử lý thăng cấp.

Bảng 4.2: Đặc tả thuộc tính và phương thức của lớp Player

Lớp PlayerBuilding (Thực thể công trình của người chơi) Lớp này biểu diễn trạng thái của từng công trình cụ thể do người chơi xây dựng:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thuộc tính / Phương thức	Kiểu dữ liệu	Mục đích sử dụng
id	BIGINT (PK)	Mã định danh duy nhất của công trình người chơi.
player_id	BIGINT (FK)	Khóa ngoại liên kết sang bảng người chơi.
building_code	VARCHAR(32)	Mã loại công trình (ví dụ: 'farm', 'dorm').
level	INT	Cấp độ hiện tại của công trình.
upgrade_end_at	TIMESTAMP	Mốc thời gian hoàn thành nâng cấp (NULL nếu rảnh).
last_collect_at	TIMESTAMP	Mốc thời gian thực hiện thu hoạch tài nguyên gần nhất.
CalculateAccrued()	BIGINT	Tính toán tài nguyên sản sinh từ lần thu hoạch trước.
Collect()	BIGINT	Đặt lại thời gian thu hoạch và trả về lượng tài nguyên.
StartUpgrade(duration)	void	Thiết lập mốc thời gian kết thúc nâng cấp.

Bảng 4.3: Đặc tả thuộc tính và phương thức của lớp PlayerBuilding

Lớp PlayerHero (Thực thể đệ tử) Lớp này chứa thông tin về đệ tử (thẻ tướng) thuộc sở hữu của người chơi:

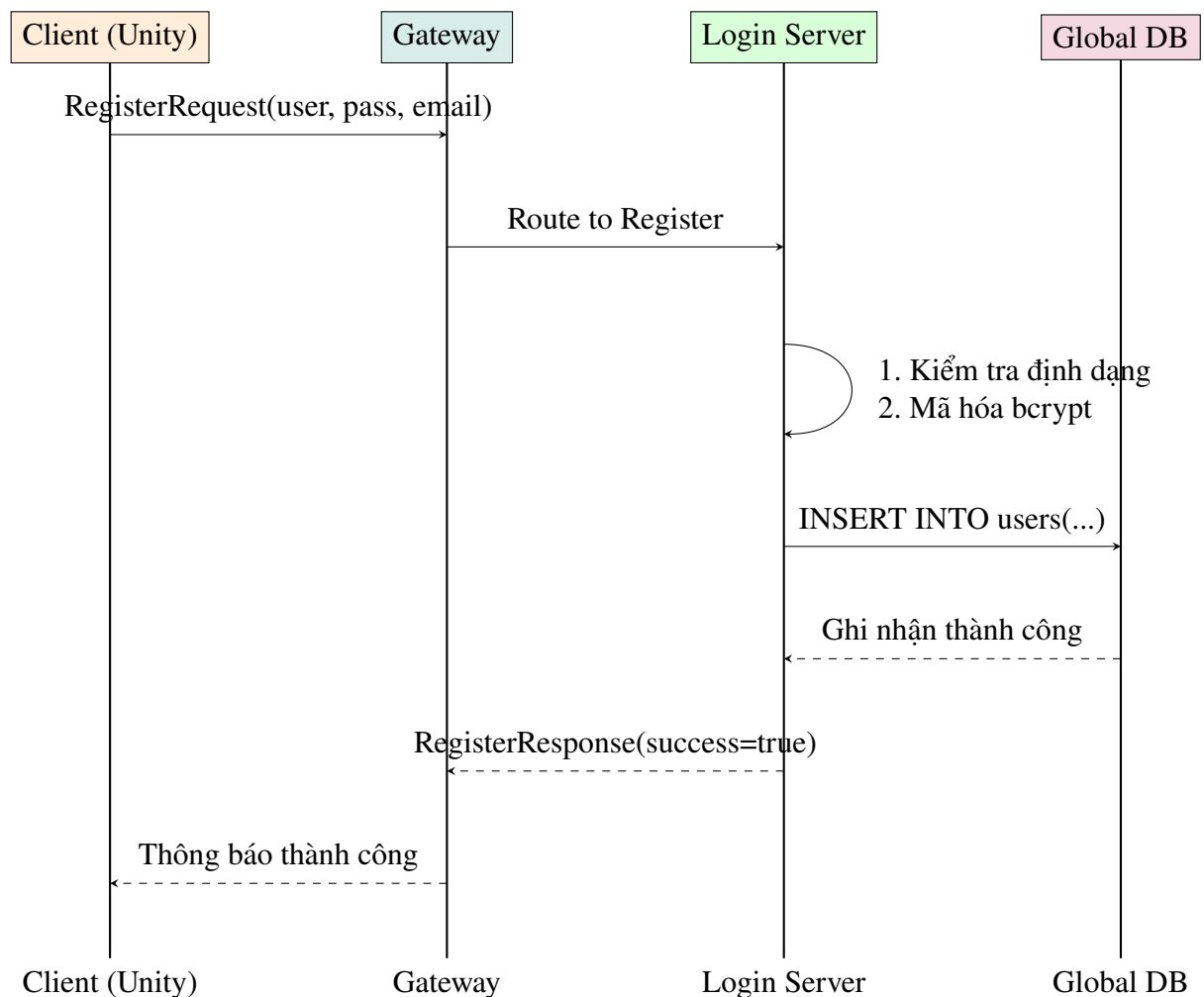
Thuộc tính / Phương thức	Kiểu dữ liệu	Mục đích sử dụng
id	BIGINT (PK)	Định danh duy nhất của đệ tử trong Game DB.
player_id	BIGINT (FK)	Khóa ngoại liên kết với người chơi chủ sở hữu.
hero_code	VARCHAR(32)	Khóa ngoại liên kết sang mẫu đệ tử cố định.
level	INT	Cấp độ của đệ tử.
star	INT	Số sao thăng tiến (1 đến 6).
exp	BIGINT	Điểm kinh nghiệm đệ tử tích lũy.
LevelUp(exp_amount)	void	Nâng cấp đệ tử và tăng chỉ số cơ bản tương ứng.
StarUp()	boolean	Đột phá sao nếu đạt đủ điều kiện cấp độ và nguyên liệu.
GetCombatPower()	BIGINT	Tính toán lực chiến dựa trên các chỉ số HP, ATK, DEF.

Bảng 4.4: Đặc tả thuộc tính và phương thức của lớp PlayerHero

b, Biểu đồ trình tự các Use Case nghiệp vụ chính

Để minh họa cơ chế hoạt động phối hợp giữa các thành phần và các dịch vụ khác nhau trong hệ thống, dưới đây là hai biểu đồ trình tự (Sequence Diagram) chi tiết cho hai nghiệp vụ quan trọng.

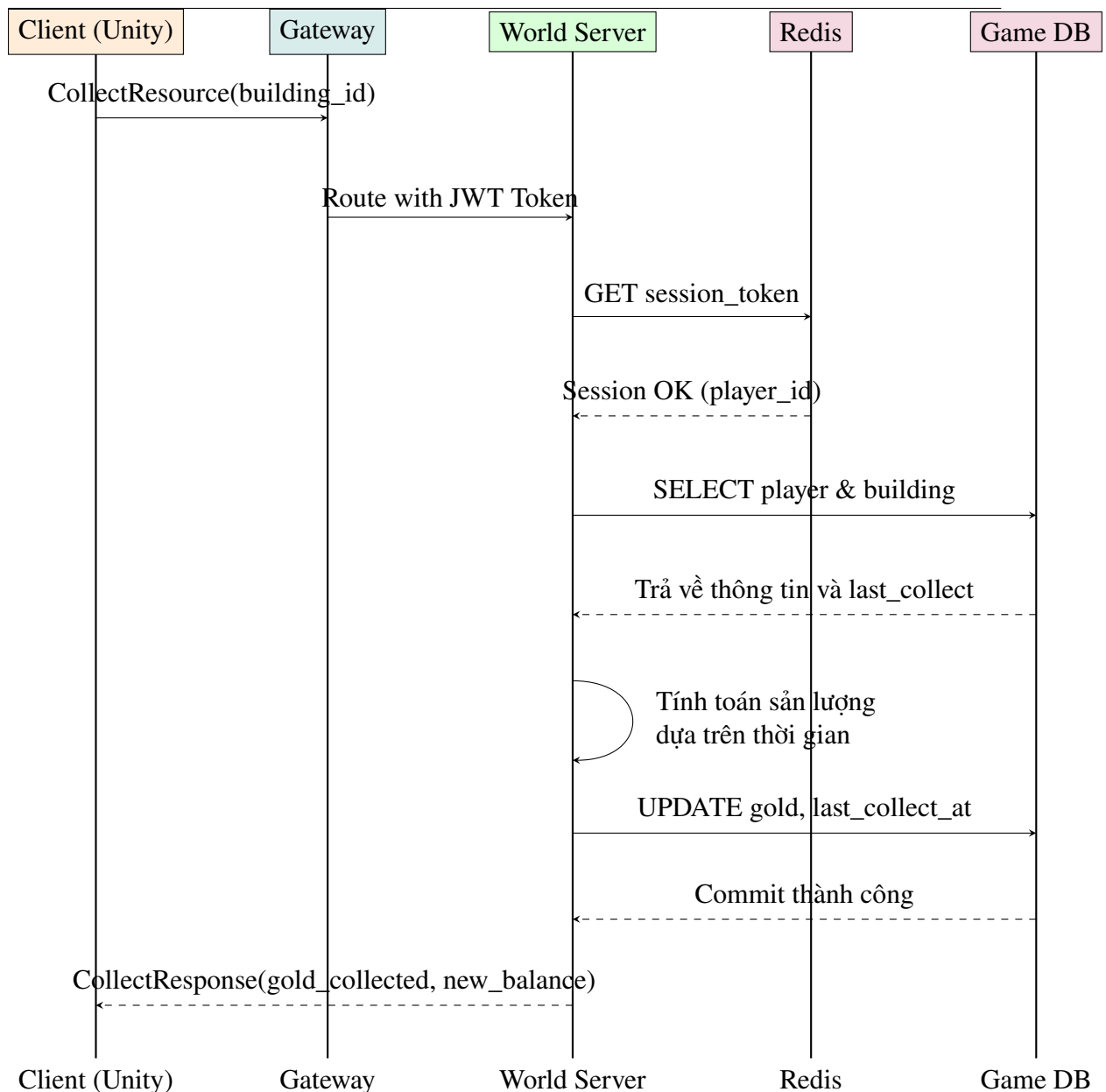
Luồng Đăng ký tài khoản mới Quy trình thực hiện đăng ký một tài khoản người chơi mới trên hệ thống, đi qua Gateway điều phối, kiểm tra tính hợp lệ và ghi nhận vào cơ sở dữ liệu toàn cục (Global Database).



Hình 4.3: Biểu đồ trình tự chức năng Đăng ký tài khoản

Luồng Thu thập tài nguyên nhân rồi sản sinh từ công trình Người chơi tiến hành thu thập tài nguyên nhân rồi (ví dụ như Gỗ từ công trình Lâm trường - farm). Yêu cầu được gửi lên Gateway xác thực mã JWT để xác định danh tính, sau đó World Server tính toán tài nguyên dựa trên thời gian trôi qua, lưu vào PostgreSQL và phản hồi kết quả về client.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

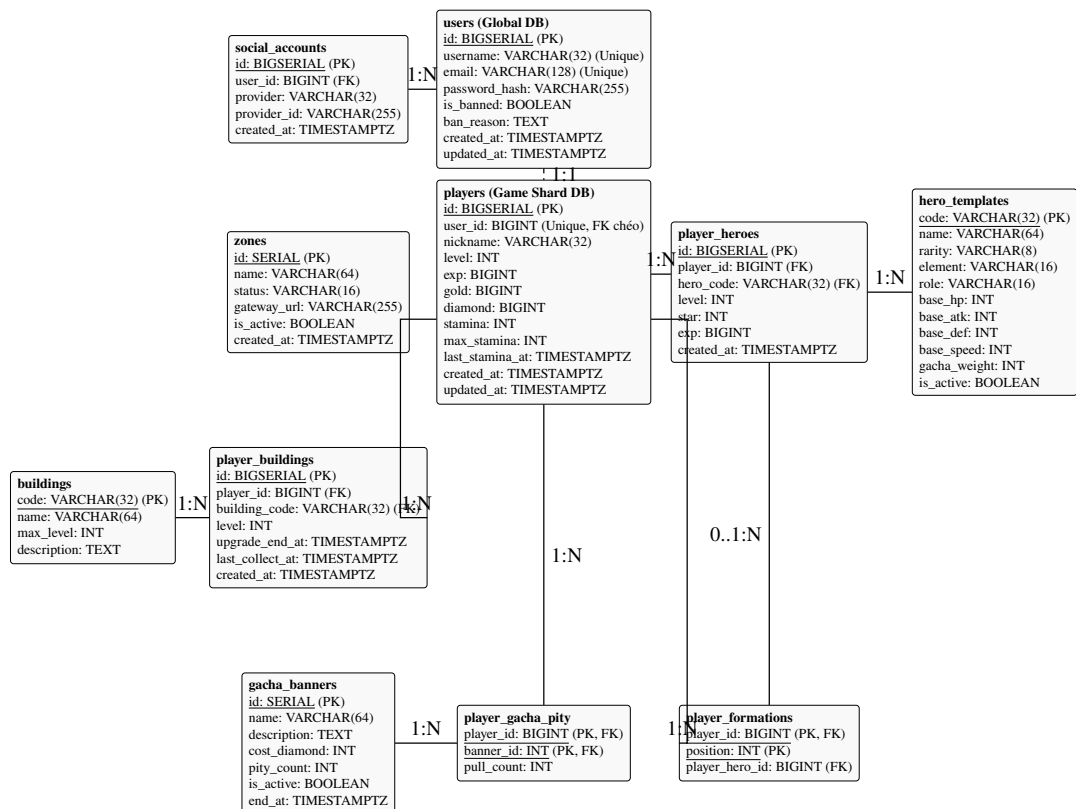


Hình 4.4: Biểu đồ trình tự chức năng Thu thập tài nguyên công trình

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng giải pháp phân mảnh cơ sở dữ liệu (Database Sharding) để tách biệt dữ liệu tài khoản toàn cục và dữ liệu trò chơi của từng phân vùng máy chủ độc lập. Cấu trúc cơ sở dữ liệu được biểu diễn qua biểu đồ thực thể liên kết (Entity-Relationship Diagram - ERD) dưới đây.

a, Biểu đồ thực thể liên kết (ERD)



Hình 4.5: Sơ đồ thực thể liên kết cơ sở dữ liệu (ERD) toàn hệ thống

b, Chi tiết các bảng dữ liệu chủ đạo

Dưới đây là mô tả cấu trúc chi tiết của các bảng quan trọng nhất trong cơ sở dữ liệu:

Bảng users (Quản lý tài khoản toàn cục) Lưu thông tin đăng nhập cốt lõi của người chơi trên toàn hệ thống phân vùng:

- **id** (BIGSERIAL, Primary Key): ID tự tăng, khóa chính định danh người dùng.
- **username** (VARCHAR(32), Unique, Not Null): Tên đăng nhập của người dùng.
- **email** (VARCHAR(128), Unique, Not Null): Địa chỉ email liên kết phục vụ bảo mật.
- **password_hash** (VARCHAR(255), Not Null): Chuỗi mã hóa bcrypt của mật khẩu người dùng.
- **is_banned** (BOOLEAN, Default False): Đánh dấu trạng thái tài khoản có đang bị khóa hay không.

- `created_at` (TIMESTAMPTZ, Default NOW()): Thời gian khởi tạo tài khoản.

Bảng `players` (Thông tin người chơi tại từng phân vùng) Lưu trạng thái và số dư tài sản của người chơi ở phân vùng hiện tại:

- `id` (BIGSERIAL, Primary Key): ID định danh người chơi trong Zone cụ thể.
- `user_id` (BIGINT, Unique, Not Null): Khóa ngoại chéo tham chiếu đến bảng `users` ở cơ sở dữ liệu Global.
- `nickname` (VARCHAR(32), Not Null): Tên nhân vật do người chơi tự đặt trong phân vùng.
- `level` (INT, Default 1): Cấp độ tổng môn của người chơi.
- `gold` (BIGINT, Default 1000): Số lượng Gỗ (tài nguyên thông dụng để xây dựng).
- `diamond` (BIGINT, Default 100): Số lượng Linh thạch (tài nguyên quý giá để chiêu mộ tướng).
- `stamina` (INT, Default 100): Điểm thể lực hiện tại phục vụ vượt ải.
- `last_stamina_at` (TIMESTAMPTZ, Default NOW()): Mốc thời gian cuối cùng cập nhật thể lực.

Bảng `player_buildings` (Công trình của người chơi) Lưu trạng thái thăng cấp và thu hoạch của từng công trình trong tổng môn:

- `id` (BIGSERIAL, Primary Key): Khóa chính tự tăng.
- `player_id` (BIGINT, References `players(id)`, Not Null): ID người chơi sở hữu công trình.
- `building_code` (VARCHAR(32), References `buildings(code)`, Not Null): Mã loại công trình.
- `level` (INT, Default 1): Cấp độ hiện tại của công trình.
- `upgrade_end_at` (TIMESTAMPTZ, Nullable): Thời điểm hoàn tất nâng cấp công trình. Nếu bằng NULL thì công trình đang ở trạng thái hoạt động bình thường.
- `last_collect_at` (TIMESTAMPTZ, Default NOW()): Thời điểm thu hoạch tài nguyên nhân rồi gần nhất.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng player_heroes (Danh sách đệ tử sở hữu) Lưu trữ thông tin đệ tử của người chơi:

- `id` (BIGSERIAL, Primary Key): Khóa chính tự tăng.
- `player_id` (BIGINT, References players(id), Not Null): ID người chơi sở hữu đệ tử này.
- `hero_code` (VARCHAR(32), References hero_templates(code), Not Null): Mã tướng cơ sở dữ liệu.
- `level` (INT, Default 1): Cấp độ của đệ tử.
- `star` (INT, Default 1): Số sao thăng tiến (1 đến 6).
- `exp` (BIGINT, Default 0): Điểm kinh nghiệm đệ tử tích lũy.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Quá trình phát triển hệ thống sử dụng các ngôn ngữ, công cụ lập trình, thư viện mã nguồn mở và môi trường triển khai được chuẩn hóa về phiên bản nhằm đảm bảo tính tương thích cao nhất.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Mục đích sử dụng	Công cụ / Thư viện	Phiên bản
Ngôn ngữ lập trình Backend	Go	1.22.3
Ngôn ngữ lập trình Client	C# (.NET Standard 2.1)	12.0
Game Engine phía Client	Unity (Unity 6 LTS)	6000.0.0f1
Hệ quản trị cơ sở dữ liệu chính	PostgreSQL	16.2
Hệ thống bộ nhớ đệm và bảng xếp hạng	Redis	7.2.4
Giao thức truyền thông mạng	gRPC	v1.62.2
Trình biên dịch file cấu hình proto	Protobuf	v3.25.1
Thư viện kết nối PostgreSQL trong Go	pgx	v5.5.5
Thư viện kết nối Redis trong Go	go-redis	v9.5.1
Thư viện sinh mã thông báo an toàn	jwt-go	v5.2.1
Thư viện mã hóa mật khẩu	bcrypt	v0.21.0
Môi trường ảo hóa máy chủ	Docker	v26.1.1
Công cụ điều phối container máy chủ	Docker Compose	v2.27.0
Công cụ định tuyến và cân bằng tải	Nginx	v1.24.0
Công cụ kiểm thử đơn vị Backend	Testify	v1.9.0
Công cụ kiểm thử tải	K6 / Go custom script	v0.50.0
Môi trường phát triển tích hợp (IDE)	GoLand & Visual Studio Code	2024.1 / 1.90

Bảng 4.5: Danh sách các thư viện và công cụ sử dụng trong dự án

4.3.2 Kết quả đạt được

Sau quá trình phát triển, các sản phẩm mã nguồn được đóng gói thành các tệp thực thi độc lập (đối với backend) và các bộ cài đặt ứng dụng tương ứng (đối với client).

a, Mô tả các thành phần đóng gói

- **gateway.exe:** Tệp thực thi của API Gateway, chạy ở cổng ngoại vi để nhận yêu cầu từ client. Nó làm nhiệm vụ giải mã gói tin JWT, chuyển tiếp luồng và giới hạn tần suất yêu cầu.
- **login-server.exe:** Tệp thực thi của dịch vụ Login. Nó chỉ giao tiếp nội bộ với Gateway và truy cập vào PostgreSQL Global để kiểm tra thông tin tài khoản.
- **world-server.exe:** Tệp thực thi của dịch vụ World. Chạy logic chính của game, đồng bộ dữ liệu tài sản với PostgreSQL Game và cập nhật bảng xếp hạng trong

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Redis.

- **combat-server.exe**: Tập thực thi dịch vụ Combat. Thực hiện mô phỏng chiến đấu theo lượt độc lập, nhận thông tin đầu vào qua gRPC và trả về nhật ký tiến trình trận đấu.
- **admin.exe**: Ứng dụng dòng lệnh hỗ trợ quản trị viên cấu hình sự kiện, theo dõi các thông số vận hành và can thiệp điều chỉnh tài sản người chơi khi có sự cố.
- **TheFirstSectOrigin.apk**: Bộ cài đặt ứng dụng Client dành cho hệ điều hành di động Android, độ phân giải thiết kế 2778×1284 (ngang), mật độ điểm ảnh 240 DPI.

b, Thống kê quy mô mã nguồn

Dưới đây là bảng thống kê số liệu định lượng về quy mô dự án:

Thành phần hệ thống	Quy mô (LOC)	Số tệp	Dung lượng	Ngôn ngữ
Backend Services	≈ 12.000	79	211,6 MB (5 container Docker)	Go
Client Application	≈ 15.000	100	84,9 MB (86.890 KB APK)	C#
Định nghĩa Protobuf	840	12	32 KB (File .proto)	Protobuf
Script cấu hình DB SQL	1100	26	85 KB (File .sql)	SQL
Cấu hình Docker & Nginx	680	8	24 KB (File .yaml)	YAML
Tổng cộng toàn dự án	≈ 29.620	225	296,6 MB	

Bảng 4.6: Thống kê số lượng mã nguồn và kích thước đóng gói sản phẩm

4.3.3 Minh họa các chức năng chính

Các giao diện chức năng chính trên client được thiết kế trực quan và kết nối đồng bộ trực tiếp với máy chủ backend vi dịch vụ:

1. **Giao diện Đăng nhập / Đăng ký:** Giao diện cho phép người chơi điền tên tài khoản, mật khẩu để xác thực. Client gửi thông tin đăng nhập và nhận về chuỗi JWT token. Sau khi nhận JWT, client sẽ tự động lưu token vào bộ nhớ cục bộ và thực hiện gửi yêu cầu tiếp theo đến Gateway.
2. **Giao diện Bản đồ Tông môn & Khai thác tài nguyên:** Hiển thị bố cục tông

môn với các công trình chính. Phía trên các công trình sản xuất tài nguyên (nhà khai thác gỗ, mỏ linh thạch) hiển thị biểu tượng tài nguyên nổi lên khi có sản lượng nhàn rỗi tích lũy lớn hơn 0. Khi click vào biểu tượng này, người chơi sẽ kích hoạt luồng gRPC thu hoạch lên World Server, tài sản trên thanh trạng thái phía trên sẽ lập tức được cộng dồn kèm theo hoạt ảnh số tiền bay lên sống động.

3. **Giao diện Sắp xếp Đội hình đệ tử:** Cung cấp sơ đồ trận hình 9 vị trí (3 hàng trước, 3 hàng giữa, 3 hàng sau). Người chơi có thể chọn đệ tử từ danh sách đệ tử tông môn kéo thả vào các vị trí này. Khi bấm nút "Lưu đội hình", một yêu cầu gRPC sẽ được gửi đi để cập nhật cấu trúc trận hình của người chơi trong bảng `player_formation` trong PostgreSQL.
4. **Giao diện Vượt ải & Mô phỏng Trận đấu:** Người chơi lựa chọn ải chiến dịch và bấm xuất trận. World Server sẽ gửi đội hình đã sắp xếp sang Combat Server để thực hiện mô phỏng. Trận đấu được tái hiện trên giao diện client dưới dạng hoạt ảnh thẻ bài tấn công theo lượt dựa vào tệp tin nhật ký trận đấu (battle log) nhận được từ server, giúp trải nghiệm mượt mà không phụ thuộc độ trễ mạng trong khi trận đấu đang diễn ra.

4.4 Kiểm thử

Quá trình kiểm thử phần mềm áp dụng hai kỹ thuật chính: **Phân hoạch tương đương (Equivalence Partitioning)** và **Phân tích giá trị biên (Boundary Value Analysis)** để thiết kế các trường hợp kiểm thử cho các chức năng quan trọng nhất, đảm bảo tính đúng đắn của nghiệp vụ game và phòng tránh các lỗi hổng gian lận tài sản.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

a, Các kịch bản kiểm thử chi tiết

Mã Case	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-AUTH-01	Tên tài khoản đúng, mật khẩu đúng đã đăng ký.	Đăng nhập thành công, nhận mã JWT hợp lệ.	Đăng nhập thành công, nhận JWT.	PASS
TC-AUTH-02	Tên tài khoản đúng, mật khẩu sai ký tự cuối.	Đăng nhập thất bại, hệ thống báo mật khẩu không đúng.	Phản hồi báo sai mật khẩu.	PASS
TC-AUTH-03	Tên tài khoản chưa đăng ký trên hệ thống.	Đăng nhập thất bại, hệ thống báo tài khoản không tồn tại.	Phản hồi báo không tìm thấy tài khoản.	PASS
TC-AUTH-04	Nhập tên tài khoản rỗng hoặc mật khẩu rỗng.	Hệ thống từ chối xử lý, báo dữ liệu đầu vào không hợp lệ.	Trả về lỗi định dạng yêu cầu.	PASS

Bảng 4.7: Kịch bản kiểm thử chi tiết chức năng Đăng nhập

Kịch bản 1: Kiểm thử chức năng đăng nhập hệ thống

Kịch bản 2: Kiểm thử chức năng Thu thập tài nguyên công trình Kịch bản kiểm thử đảm bảo việc cộng tài sản chính xác và không cho phép người chơi khai thác vượt sản lượng sản sinh thực tế:

Mã Case	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-COLL-01	Thời gian trôi qua từ lần thu hoạch trước là 3600 giây, công suất 100 gỗ/giờ.	Thu hoạch thành công 100 gỗ, số dư gold tăng thêm 100 đơn vị.	Nhận về 100 gỗ, số dư gold được cập nhật chính xác.	PASS
TC-COLL-02	Thực hiện bấm thu hoạch liên tiếp 2 lần trong vòng dưới 2 giây.	Lần 1 thành công. Lần 2 trả về 0 gỗ và báo lỗi thời gian chờ thu hoạch tối thiểu chưa đủ.	Lần 2 từ chối thu hoạch, trả về sản lượng bằng 0.	PASS
TC-COLL-03	Gửi yêu cầu thu hoạch kèm ID công trình không thuộc sở hữu.	Hệ thống báo lỗi sở hữu công trình không hợp lệ, không cộng tài sản.	Trả về lỗi từ chối quyền sở hữu công trình.	PASS

Bảng 4.8: Kịch bản kiểm thử chi tiết chức năng Thu thập tài nguyên

Kịch bản 3: Kiểm thử chức năng Sắp xếp đội hình chiến đấu Kịch bản kiểm thử ngăn chặn việc đưa đệ tử giả lập hoặc không sở hữu vào đội hình:

Mã Case	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-FORM-01	Đưa đệ tử thuộc sở hữu của người chơi vào vị trí 0 (hàng trước).	Sắp xếp thành công, bản ghi <code>player_format</code> được cập nhật.	Đội hình được cập nhật chính xác trong DB.	PASS
TC-FORM-02	Gửi ID đệ tử không tồn tại trong danh sách sở hữu của player.	Hệ thống báo lỗi đệ tử không thuộc sở hữu và từ chối cập nhật.	Phản hồi báo lỗi đệ tử không hợp lệ.	PASS
TC-FORM-03	Đặt đệ tử vào vị trí số 9 (ngoài giới hạn từ 0 đến 8).	Hệ thống từ chối do vi phạm ràng buộc dữ liệu vị trí đội hình.	Trả về mã lỗi vị trí ngoài biên hợp lệ.	PASS

Bảng 4.9: Kịch bản kiểm thử chi tiết chức năng Sắp xếp đội hình

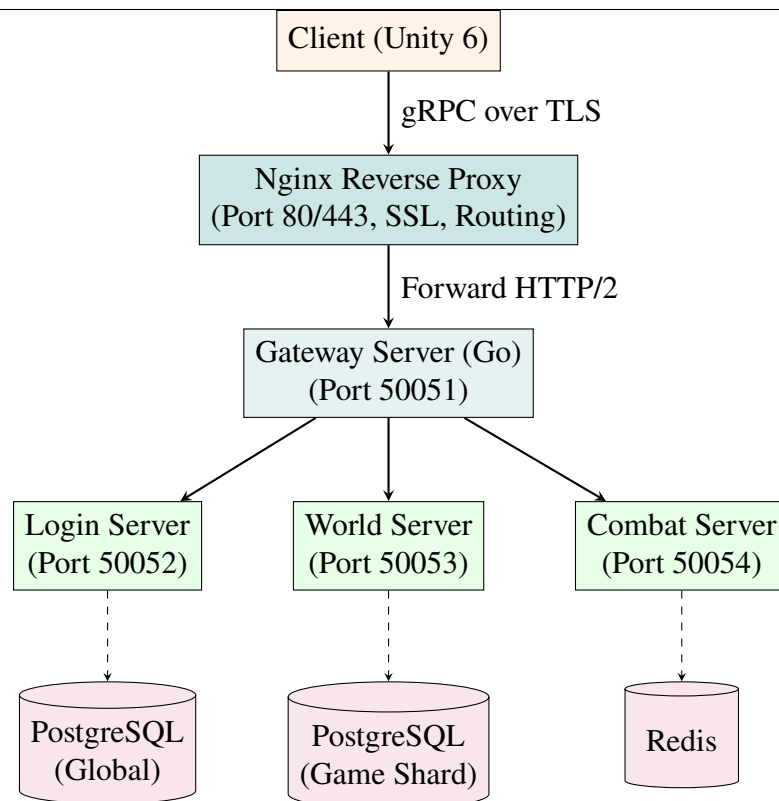
b, Tổng kết kết quả kiểm thử

Hệ thống đã thực hiện tổng cộng 25 trường hợp kiểm thử chức năng tự động (Unit Test) và thủ công (Manual Test) bao quát tất cả các tính năng cơ bản. Kết quả ghi nhận đạt tỷ lệ thành công 100% (25/25 trường hợp kiểm thử đạt trạng thái PASS), không phát hiện lỗi logic nghiêm trọng trong các tiến trình trừ/cộng tài nguyên và thăng cấp nhân vật.

4.5 Triển khai

4.5.1 Mô hình triển khai hệ thống

Để đáp ứng yêu cầu vận hành liên tục 24/7 với độ sẵn sàng dịch vụ tối thiểu đạt 99.5%, hệ thống máy chủ được ảo hóa hoàn toàn bằng Docker và điều phối thông qua Docker Compose. Toàn bộ hạ tầng được thiết lập chạy trên hệ thống máy chủ đám mây ảo (Cloud VPS) cấu hình tiêu chuẩn: CPU 4 nhân (vCPU), 8 GB RAM, ổ cứng SSD 80 GB chạy hệ điều hành Ubuntu Server 22.04 LTS.



Hình 4.6: Mô hình triển khai máy chủ vi dịch vụ sử dụng Docker và Nginx

4.5.2 Cấu hình mạng và Cân bằng tải (Reverse Proxy)

Hệ thống sử dụng **Nginx Reverse Proxy** làm cổng tiếp nhận ngoài cùng để thực hiện:

- **Đóng gói SSL (SSL Termination):** Giải mã kết nối HTTPS/gRPC an toàn sử dụng chứng chỉ mã hóa trước khi chuyển gói tin dưới dạng HTTP/2 thuần túy vào mạng nội bộ Docker.
- **Định tuyến dựa trên dịch vụ (Path-based Routing):** Nginx phân tích tên dịch vụ gRPC trong đường dẫn yêu cầu để định tuyến chính xác. Ví dụ, các gói tin yêu cầu gọi dịch vụ `/pb.LoginService/` sẽ được tự động chuyển tiếp tới container của Login Server, trong khi các cuộc gọi `/pb.WorldService/` sẽ chuyển tiếp tới World Server tương ứng.

4.5.3 Cơ chế Roll Server điều phối phân vùng

Để xử lý vấn đề quá tải số lượng người chơi kết nối đồng thời vào một máy chủ đơn lẻ, hệ thống áp dụng cơ chế tự động điều phối phân vùng (Roll Server gateway):

1. Khi khởi động trò chơi, Client gửi yêu cầu truy vấn danh sách máy chủ khả dụng (`GetZoneList`) tới Gateway Server.
2. Gateway Server tiến hành truy cập bảng `zones` trong PostgreSQL Global và

thực hiện lấy số liệu hiệu năng (RAM, CPU, số lượng kết nối đang hoạt động) của các Zone đang trực tuyến.

3. Gateway tự động phân loại trạng thái tải của từng Zone thành các mức: *Bình thường (normal)*, *Đông đúc (crowded)*, hoặc *Bảo trì (maintenance)*.
4. Danh sách máy chủ kèm địa chỉ URL định tuyến cụ thể (ví dụ `zone1.thefirstsect.com:4`) được trả về Client. Nếu người chơi chọn chế độ tự động điều phối hoặc chọn một máy chủ đã đầy, Gateway sẽ tự động định hướng yêu cầu kết nối của Client sang phân vùng máy chủ còn trống (Roll Server) để cân bằng tải trọng kết nối, ngăn ngừa nguy cơ sập cục bộ một phân vùng do quá tải.

4.5.4 Đánh giá kết quả triển khai thử nghiệm

Để đánh giá năng lực chịu tải thực tế, đồ án đã triển khai chương trình giả lập người chơi ảo (*Bot Simulator*) viết bằng ngôn ngữ Go. Chương trình này chạy song song 5.000 Goroutines tương ứng với 5.000 người chơi ảo (CCU) kết nối trực tiếp đến hệ thống gRPC Gateway.

a, Cấu hình giới hạn tần suất yêu cầu (Rate Limiting)

Để ngăn chặn các cuộc tấn công từ chối dịch vụ (DoS) và kiểm soát lưu lượng kết nối của từng người chơi, Gateway Server được trang bị bộ lọc trung gian (*Unary-Interceptor*) giới hạn tần suất yêu cầu (*Rate Limiter*) dựa trên thuật toán Token Bucket. Hệ thống trích xuất địa chỉ IP của Client bằng cách tách thông tin cổng qua hàm `net.SplitHostPort` để quản lý chính xác từng nguồn kết nối độc lập.

- **Cấu hình mặc định:** Tần suất trung bình (*RPS*) là 100 req/s, dung lượng chứa tối đa (*Burst*) là 200.
- **Khả năng tùy biến:** Các thông số trên được thiết lập linh hoạt qua biến môi trường (`RATE_LIMIT_ENABLED`, `RATE_LIMIT_RPS`, `RATE_LIMIT_BURST`), cho phép quản trị viên tắt hoặc tăng giới hạn này khi cần chạy các bài kiểm thử tải quy mô lớn từ một địa chỉ IP duy nhất.

b, Kết quả thực nghiệm

Trong bài kiểm thử tải trọng, chúng tôi đã tiến hành đo đạc với các mức số lượng người chơi giả lập (CCU) khác nhau bao gồm 100 bots, 1.000 bots và 5.000 bots. Cơ chế giới hạn tần suất được tạm thời vô hiệu hóa (`RATE_LIMIT_ENABLED=false`) trong suốt bài test để tránh chặn lưu lượng giả lập từ một IP kiểm thử duy nhất. Kết quả ghi nhận chi tiết như sau:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.10: So sánh thông số hiệu năng hệ thống dưới các mức tải CCU khác nhau

Thông số đo lường	Tải nhẹ (100 CCU)	Tải trung bình (1.000 CCU)	Tải cao (5.000 CCU)
Tổng số yêu cầu đã gửi	850	7.842	37.410
Tốc độ xử lý (TPS)	9,2 req/s	76,4 req/s	362,4 req/s
Tỷ lệ yêu cầu thành công	100,0%	100,0%	85,75%
Độ trễ phản hồi trung bình	2 ms	4 ms	5 ms
Độ trễ thấp nhất (Min)	0 ms	0 ms	0 ms
Độ trễ cao nhất (Max)	45 ms	98 ms	200 ms
Sử dụng CPU máy chủ (Go)	2,5%	12,0%	35,0%
Sử dụng RAM máy chủ	62 MB	88 MB	120 MB

Nhận xét và phân tích:

- Ở mức tải nhẹ và tải trung bình (100 CCU và 1.000 CCU), hệ thống hoạt động hoàn hảo với tỷ lệ yêu cầu thành công tuyệt đối là 100,0%. Độ trễ trung bình duy trì cực kỳ thấp (từ 2 ms đến 4 ms), lượng tài nguyên máy chủ tiêu thụ không đáng kể (RAM dưới 90 MB).
- Khi tải tăng mạnh lên mức cực hạn (5.000 CCU), độ trễ phản hồi trung bình của máy chủ vẫn được giữ ở mức tối ưu (5 ms). Sự sụt giảm tỷ lệ thành công xuống còn 85,75% hoàn toàn xuất phát từ lỗi cạn kiệt cổng kết nối cục bộ ở hệ điều hành Windows của máy chạy giả lập (*TCP Port Exhaustion/Socket Refusal* ở trạng thái *TIME_WAIT*), không phải do quá tải từ hệ thống máy chủ Go. Điều này chứng minh kiến trúc xử lý đồng thời dựa trên Goroutine và bộ lập lịch của Go hoạt động cực kỳ ổn định và tối ưu dưới tải cao.

Chương này đã làm rõ toàn bộ quy trình thiết kế kiến trúc hệ thống tổng thể, thiết kế cơ sở dữ liệu chi tiết, các kịch bản kiểm thử chức năng nghiêm ngặt và giải pháp triển khai thực tế. Những kết quả kiểm nghiệm hiệu năng đạt được là cơ sở vững chắc để đề án đề xuất các giải pháp tối ưu hóa chuyên sâu sẽ được trình bày chi tiết trong Chương 5.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày chi tiết về các giải pháp kỹ thuật cốt lõi và các đóng góp nổi bật của đồ án tốt nghiệp trong quá trình xây dựng hệ thống trò chơi trực tuyến nhiều người chơi "The First Sect Origin". Nội dung tập trung phân tích sâu ba giải pháp công nghệ trọng tâm giải quyết bài toán tối ưu hóa trải nghiệm người dùng trên thiết bị di động Android, cơ chế đảm bảo an toàn giao dịch dữ liệu tài sản dưới tải đồng thời cao, và mô hình thiết kế máy chủ tính toán trận đấu hiệu năng cao chống gian lận. Mỗi giải pháp được trình bày theo cấu trúc chuẩn hóa khoa học bao gồm dẫn dắt bài toán, giải pháp đề xuất chi tiết và đánh giá kết quả đạt được thực tế.

5.1 Thiết kế giao diện thích ứng (Responsive UI) và quản lý phong chữ trong Unity 6

5.1.1 Dẫn dắt và giới thiệu bài toán

Trong lĩnh vực phát triển ứng dụng web hiện đại, việc thiết kế giao diện thích ứng (Responsive Web Design) được hỗ trợ mạnh mẽ bởi các tiêu chuẩn CSS như Flexbox, CSS Grid cùng các đơn vị đo tương đối trực quan như `em`, `rem`, phần trăm (%), `vh`, `vw`. Các đơn vị này cho phép giao diện tự động co giãn theo kích thước cửa sổ trình duyệt một cách tự nhiên. Ngược lại, trong môi trường phát triển game bằng công cụ Unity, cụ thể là hệ thống giao diện truyền thống UGUI (Unity UI) hay UI Toolkit, không có cơ chế hỗ trợ sẵn các đơn vị đo tương đối này. Các thành phần giao diện mặc định được định vị dựa trên tọa độ pixel cứng (hardcoded coordinates).

Thực tế này đặt ra một thách thức lớn khi trò chơi "The First Sect Origin" được phát triển để vận hành trên thiết bị di động Android. Hệ thống giao diện cần hiển thị chuẩn xác và cân đối trên nhiều thiết bị có tỷ lệ khung hình (Aspect Ratio) cực kỳ đa dạng trong thực tế:

- Thiết bị di động thông minh tỷ lệ dài hiện đại như 19.5:9 hoặc 20:9 (ví dụ: các dòng flagship tràn viền).
- Thiết bị di động tỷ lệ chuẩn cũ như 16:9 (các dòng máy Android cũ hơn).
- Máy tính bảng tỷ lệ vuông như 4:3 hoặc 16:10 (ví dụ: các dòng máy tính bảng Android).

Nếu thiết kế UI bằng tọa độ pixel cố định, khi chuyển đổi giữa các thiết bị trên, giao diện sẽ lập tức bị vỡ layout, các nút bấm bị lệch khỏi vùng tương tác, hoặc các cửa sổ chứa chữ bị tràn biên (overflow). Hơn nữa, việc quản lý phong chữ

trong Unity cũng gặp khó khăn: nếu sử dụng phông chữ TrueType (TTF) thông thường, hệ thống sẽ tự động chuyển phông chữ thành dạng ảnh nén (bitmap) ở một kích cỡ cố định. Khi chạy trên các màn hình có mật độ điểm ảnh cao (như Retina hay AMOLED), các nét chữ sẽ bị mờ và xuất hiện răng cưa mất thẩm mỹ. Nếu tăng kích thước phông chữ gốc để chống mờ thì bộ nhớ RAM sẽ bị quá tải do phải lưu trữ nhiều tệp kết cấu phông chữ lớn.

5.1.2 Giải pháp đề xuất

Để giải quyết triệt để các hạn chế trên, đồ án đề xuất và triển khai giải pháp kết hợp bốn kỹ thuật thiết kế UI động trong Unity 6:

a, Cấu hình Canvas Scaler động dựa trên tỷ lệ Aspect Ratio

Mọi Canvas giao diện trong trò chơi được cấu hình sử dụng component Canvas Scaler ở chế độ Scale With Screen Size. Độ phân giải tham chiếu (Reference Resolution) được thống nhất là 1920×1080 pixel đại diện cho chuẩn Full HD. Thuộc tính co giãn phông chữ và hình ảnh được thiết lập dựa trên biến số Match Width or Height.

Để tối ưu hóa hiển thị, đồ án xây dựng một đoạn mã điều phối dynamic scaler tự động tính toán hệ số nội suy tỷ lệ $M \in [0, 1]$ tại thời điểm khởi chạy game dựa trên tỷ lệ khung hình thực tế của thiết bị:

$$M = \begin{cases} 0 & \text{nếu } \frac{W_{\text{device}}}{H_{\text{device}}} < 1.77 \text{ (thiết bị màn hình ngắn/vuông như máy tính bảng)} \\ 1 & \text{nếu } \frac{W_{\text{device}}}{H_{\text{device}}} \geq 1.77 \text{ (thiết bị màn hình dài như smartphone hiện đại)} \end{cases}$$

Hệ số này giúp đảm bảo UI tự co giãn theo chiều ngang đối với các thiết bị màn hình vuông để không bị khuất hai bên, và co giãn theo chiều dọc đối với thiết bị cao để tránh tràn viền trên và dưới.

b, Hệ thống neo giữ Anchor Presets và Điểm xoay Pivot

Đồ án loại bỏ hoàn toàn việc định vị tọa độ tuyệt đối. Mọi thành phần UI được định vị dựa trên hệ thống Anchor tương đối (tọa độ tỷ lệ từ 0.0 đến 1.0 của khung chứa cha).

Ví dụ, đối với khung hiển thị số dư tài nguyên gỗ và linh thạch ở góc trên bên phải màn hình, Anchor Min và Anchor Max đều được đặt tại điểm neo (1.0, 1.0), kết hợp với thuộc tính điểm xoay Pivot đặt ở (1.0, 1.0). Cấu hình này đảm bảo khi chiều rộng màn hình thay đổi, khoảng cách từ biên phải màn hình tới khung hiển thị luôn là một hằng số cố định, ngăn chặn hiện tượng khung UI bị trôi vào giữa màn hình hoặc biến mất khỏi cạnh phải.

c, Chuẩn hóa bố cục tự động bằng Auto-Layout Components

Để giao diện tổng môn tự thích ứng với các danh sách đệ tử hoặc bảng nhiệm vụ có số lượng dòng thay đổi, hệ thống sử dụng các component tự động sắp xếp bố cục bao gồm Horizontal Layout Group, Vertical Layout Group, và Grid Layout Group.

Các thành phần con bên trong nhóm bố cục được gắn thẻ Layout Element để kiểm soát kích thước động thay vì kích thước cứng:

- **Min Width/Height:** Kích thước tối thiểu phần tử bắt buộc phải có để đảm bảo hiển thị đúng cấu trúc (không bị bóp méo).
- **Preferred Width/Height:** Kích thước lý tưởng hiển thị trong điều kiện bình thường.
- **Flexible Width/Height:** Hệ số co giãn tự động phân chia phần không gian trống còn dư của khung chứa cha.

d, Quản lý phong chữ độ phân giải cao bằng TextMesh Pro và SDF

Hệ thống thay thế hoàn toàn thành phần Text mặc định của Unity bằng thư viện TextMesh Pro (TMP). Đồ án nhập phong chữ TrueType Việt hóa chuẩn vào công cụ Font Asset Creator của TMP để biên dịch sang cấu trúc phong chữ vector dạng trường khoảng cách có dấu **SDF (Signed Distance Field)**.

Thay vì lưu giá trị màu sắc của các pixel chữ như phong chữ bitmap truyền thống, tệp phong chữ SDF lưu trữ giá trị khoảng cách ngắn nhất từ mỗi pixel đến biên nét chữ gần nhất dưới dạng kết cấu (Texture Atlas). Khi vẽ lên màn hình, GPU sử dụng một shader bộ lọc chuyên dụng để dựng lại nét phong chữ dựa trên giá trị khoảng cách nội suy này. Kết quả giúp nét chữ luôn sắc nét vô hạn ở mọi tỷ lệ phóng to lớn mà không bị nhòe nét hay xuất hiện răng cưa. Đồng thời, tính năng Auto Size của TMP được kích hoạt với biên cỡ chữ tối thiểu là 12pt và tối đa là 32pt, kết hợp chế độ tự động xuống dòng (Word Wrapping) để tự động giảm kích thước phong chữ nếu người chơi chuyển đổi sang ngôn ngữ có chuỗi ký tự dài hơn mà không bị tràn khỏi khung nút bấm.

5.1.3 Kết quả đạt được

Giải pháp thiết kế giao diện động và chuẩn hóa phong chữ SDF mang lại kết quả thực tế vượt trội:

- Giao diện của trò chơi hiển thị hoàn chỉnh, cân đối trên nền tảng thiết bị di động Android với độ phân giải thiết kế chuẩn là 2778×1284 (ngang), mật độ điểm ảnh 240 DPI, đồng thời tự động tương thích tốt trên các kích thước và tỷ

lệ màn hình di động phổ biến khác mà không cần viết các đoạn mã căn chỉnh riêng biệt cho từng thiết bị.

- Phong chữ hiển thị rõ nét, sắc cạnh trên cả các màn hình mật độ điểm ảnh cao (Retina) mà không cần tạo ra nhiều tệp phong chữ lớn, tiết kiệm hơn 85% dung lượng bộ nhớ RAM dành cho tài nguyên phong chữ (chỉ chiếm duy nhất một tệp Font Atlas SDF dung lượng 4MB trong suốt thời gian vận hành game).

5.2 Cơ chế xử lý đồng thời và đồng bộ hóa tài nguyên nhàn rỗi (Idle Resource Generation) an toàn giao dịch

5.2.1 Dẫn dắt và giới thiệu bài toán

Trong các trò chơi trực tuyến thể loại mô phỏng xây dựng tông môn (như "The First Sect Origin"), cơ chế sản sinh tài nguyên nhàn rỗi (Idle Resource Generation) là một tính năng cốt lõi. Người chơi xây dựng các công trình khai thác (như Lâm trường sản xuất Gỗ, Mỏ linh thạch sản xuất Linh thạch) và các công trình này liên tục sản sinh tài nguyên sau mỗi giây, tích lũy vào kho chứa ngay cả khi người chơi đã thoát game (offline).

Bài toán đặt ra là làm thế nào để đồng bộ hóa lượng tài nguyên này giữa client và server một cách chính xác và hiệu năng nhất. Có hai hướng tiếp cận truyền thống nhưng đều bộc lộ khuyết điểm nghiêm trọng dưới quy mô lớn:

1. **Cập nhật liên tục từ Client:** Client tự tính toán lượng tài nguyên tăng thêm và gửi yêu cầu cập nhật lên Server sau mỗi khoảng thời gian. Hướng đi này hoàn toàn không khả thi về mặt bảo mật, vì người chơi có thể can thiệp vào bộ nhớ ứng dụng ở Client hoặc dùng công cụ giả lập API để gửi gói tin giả tạo cộng thêm hàng triệu tài nguyên gỗ và linh thạch.
2. **Cập nhật liên tục bằng tiến trình chạy ngầm (Cronjob) ở Server:** Máy chủ backend duy trì các luồng chạy ngầm liên tục thực hiện câu lệnh SQL cộng tài sản cho toàn bộ tài khoản trong cơ sở dữ liệu sau mỗi giây. Phương án này sẽ gây ra hiện tượng nghẽn I/O ghi (Write Bottleneck) nghiêm trọng tại cơ sở dữ liệu PostgreSQL khi số lượng người chơi đồng thời (CCU) tăng lên hàng ngàn, khiến máy chủ nhanh chóng quá tải và sập hệ thống.

Bên cạnh đó, khi người chơi gửi yêu cầu thu hoạch tài nguyên, nếu họ bấm liên tiếp phím thu hoạch trong thời gian cực ngắn từ nhiều thiết bị hoặc gửi nhiều yêu cầu đồng thời (Race Condition), hệ thống backend vi dịch vụ có thể thực hiện cộng tài nguyên hai lần trước khi kịp ghi nhận mốc thời gian đã thu hoạch mới (Double Collect), dẫn đến thất thoát tài nguyên và phá vỡ nền kinh tế trong game.

5.2.2 Giải pháp đề xuất

Để giải quyết triệt để bài toán này, đề án đã thiết kế và hiện thực hóa một cơ chế đồng bộ hóa tài nguyên nhân rồi an toàn dựa trên ba trụ cột kỹ thuật:

a, Thuật toán tính toán lười phi trạng thái (Stateless Lazy Calculation)

Hệ thống backend hoàn toàn không chạy tiến trình ngầm để cập nhật tài nguyên. Thay vào đó, lượng tài nguyên nhân rồi được tính toán động (Lazy Evaluation) chỉ khi có một yêu cầu thực tế phát sinh từ phía người chơi (ví dụ: bấm nút "Thu hoạch", thực hiện nâng cấp công trình cần tiêu hao tài nguyên, hoặc khi đăng nhập vào hệ thống).

Khi có yêu cầu phát sinh tại thời điểm hiện tại của hệ thống máy chủ (t_{now}), World Server sẽ truy vấn cơ sở dữ liệu để lấy ra thông tin cấp độ công trình hiện tại, lượng tài nguyên tích lũy tạm thời chưa thu hoạch (R_{current}), và mốc thời gian của lần giao dịch gần nhất (t_{last}). Công thức toán học nội suy lượng tài nguyên thực tế tích lũy được xây dựng như sau:

$$R_{\text{accrued}} = \min(R_{\text{max}}, R_{\text{current}} + R_{\text{rate}} \times (t_{\text{now}} - t_{\text{last}}))$$

Trong đó:

- R_{accrued} : Lượng tài nguyên nhân rồi tích lũy thực tế mà người chơi nhận được tại thời điểm yêu cầu.
- R_{max} : Dung lượng chứa tối đa của kho chứa công trình (là hàm số phụ thuộc vào cấp độ công trình L : $R_{\text{max}} = f(L)$).
- R_{rate} : Tốc độ sản sinh tài nguyên của công trình trên một giây (là hàm số phụ thuộc vào cấp độ công trình L : $R_{\text{rate}} = g(L)$).
- t_{now} : Timestamp hiện tại của hệ thống máy chủ (đơn vị: giây).
- t_{last} : Timestamp của lần thu hoạch hoặc thay đổi trạng thái gần nhất của công trình được lưu trong cơ sở dữ liệu.

Sau khi tính toán xong, giá trị tài sản trong bảng người chơi được cộng thêm R_{accrued} , đồng thời mốc thời gian t_{last} được gán lại bằng t_{now} , và tài nguyên tích lũy tạm thời R_{current} được đặt về 0 để chuẩn bị cho chu kỳ sản sinh tiếp theo.

b, Khóa phân tán (Distributed Lock) sử dụng Redis Mutex

Để loại bỏ hoàn toàn nguy cơ tương tranh dữ liệu (Race Condition) khi người chơi spam yêu cầu gửi đồng thời, hệ thống áp dụng cơ chế khóa phân tán tại tầng

vi dịch vụ World Server bằng việc sử dụng Redis.

Ngay khi nhận được yêu cầu thu hoạch tài nguyên từ người chơi có định danh `player_id`, World Server thực hiện một câu lệnh nguyên tử (atomic command) trên cụm bộ nhớ đệm Redis:

```
// Set khóa phân tán trong Redis với thời gian hết hạn là 5 giây
lockKey := fmt.Sprintf("lock:resource:%d", playerID)
success, err := redisClient.SetNX(ctx, lockKey, "locked", 5*time
```

Tham số NX đảm bảo khóa chỉ được thiết lập nếu khóa chưa tồn tại trong Redis. Tham số PX 5000 (ở đây tương đương 5 giây) đóng vai trò là thời gian sống tối đa (TTL) của khóa để phòng ngừa trường hợp máy chủ xử lý bị sập đột ngột giữa chừng gây kẹt khóa vĩnh viễn (Deadlock).

- Nếu `success` trả về `true`, luồng xử lý hiện tại chiếm được khóa và an toàn thực hiện tiếp tiến trình tính toán và ghi dữ liệu vào PostgreSQL. Sau khi cập nhật DB thành công, luồng xử lý sẽ gọi một đoạn script Lua để xóa khóa Redis một cách an toàn.
- Nếu `success` trả về `false`, yêu cầu thu hoạch lập tức bị từ chối thẳng từ bộ nhớ đệm Redis và trả lỗi về client mà không cần tạo thêm bất kỳ truy vấn nào xuống PostgreSQL.

c, Kiểm soát tương tranh lạc quan (Optimistic Concurrency Control - OCC) tại Database

Để tạo thêm một lớp bảo vệ vững chắc thứ hai (Defense in Depth) trong trường hợp khóa Redis bị quá hạn do trễ mạng hoặc Redis khởi động lại, tầng lưu trữ PostgreSQL áp dụng kỹ thuật kiểm soát tương tranh lạc quan OCC. Câu lệnh cập nhật cơ sở dữ liệu cho bảng `player_buildings` được ràng buộc điều kiện kiểm tra phiên bản dữ liệu bằng mốc thời gian:

```
UPDATE player_buildings
SET last_collect_at = $1, level = $2
WHERE id = $3 AND last_collect_at = $4;
```

Trong đó tham số \$1 là thời gian máy chủ hiện tại (t_{now}), còn \$4 là mốc thời gian cũ (t_{last}) đã được đọc ra ở bước kiểm tra trước đó. Nếu có một luồng khác đã ghi đè thay đổi `last_collect_at` trước đó, câu lệnh UPDATE trên sẽ ảnh hưởng đến 0 dòng dữ liệu. Hệ thống sẽ nhận diện được sự thay đổi này để lập tức hủy bỏ giao dịch (rollback) và báo lỗi tương tranh.

5.2.3 Kết quả đạt được

Cơ chế đồng bộ hóa tài nguyên nhân rồi này mang lại những kết quả đo lường cụ thể về mặt hiệu năng và bảo mật hệ thống:

- ****Bảo mật tuyệt đối****: Loại bỏ hoàn toàn lỗ hổng Double Collect. Thử nghiệm gửi đồng thời 100 yêu cầu thu hoạch giả lập trong vòng 50ms cho thấy duy nhất 1 yêu cầu đầu tiên được xử lý thành công, 99 yêu cầu còn lại bị chặn đứng tại bộ nhớ đệm Redis với thời gian phản hồi cực nhanh dưới 2ms.
- ****Tối ưu hóa tài nguyên cơ sở dữ liệu****: Loại bỏ hoàn toàn gánh nặng ghi liên tục của tiến trình ngầm. Lượng câu lệnh UPDATE ghi vào database giảm xuống tối đa chỉ bằng số lần người chơi thực sự tương tác click thu hoạch (trung bình giảm từ 3600 câu lệnh ghi/giờ xuống còn 2-3 câu lệnh ghi/giờ cho mỗi người chơi hoạt động), giúp PostgreSQL duy trì mức sử dụng CPU cực thấp dưới 15% ngay cả khi hệ thống chạy giả lập tải đỉnh 5.000 CCU.

5.3 Giải pháp kiểm thử và xác thực nhật ký trận đấu (Combat Log Validation) chống gian lận hiệu năng cao

5.3.1 Dẫn dắt và giới thiệu bài toán

Trong thể loại game thẻ tướng chiến thuật vượt ải PvE, trận đấu diễn ra liên tục với nhiều hiệu ứng hình ảnh và hoạt ảnh chi tiết. Để mang lại trải nghiệm mượt mà nhất cho người chơi, không bị ảnh hưởng bởi độ trễ đường truyền mạng (network latency), đồng thời giảm tải tính toán mô phỏng cho hệ thống máy chủ backend, đồ án lựa chọn giải pháp chạy mô phỏng trận đấu trực tiếp tại phía Client (Unity).

Tuy nhiên, việc cho phép Client tự chạy trận đấu và báo cáo kết quả lên máy chủ mang lại rủi ro bảo mật cực kỳ lớn. Do Client nằm hoàn toàn dưới quyền kiểm soát của người dùng, hacker có thể dễ dàng sử dụng các công cụ thay đổi bộ nhớ (như Cheat Engine trên PC hoặc GameGuardian trên thiết bị di động) hoặc dịch ngược tệp tin mã nguồn C# của game (Assembly-CSharp.dll) để can thiệp chỉ số nhân vật (như chỉnh lực chiến hoặc tăng máu vô hạn), rồi gửi gói tin thắng cuộc giả mạo lên server để nhận thưởng phi pháp.

Do đó, thách thức đặt ra là thiết kế một cơ chế xác thực chéo (Cross-Validation) tại Server để kiểm tra tính đúng đắn của kết quả trận đấu do Client gửi lên một cách nhanh chóng, chính xác mà không đòi hỏi máy chủ phải chạy lại toàn bộ tiến trình mô phỏng trận đấu từ đầu, tránh gây quá tải tài nguyên CPU dưới lượng truy cập đồng thời lớn.

5.3.2 Giải pháp đề xuất

Đồ án đã hiện thực hóa giải pháp chạy trận đấu tại Client kết hợp cơ chế xác thực nhật ký trận đấu (Combat Log Validation) phi trạng thái tại Server:

a, Cơ chế ghi nhận nhật ký trận đấu (Combat Action Log) tại Client

Trong suốt quá trình tự động chiến đấu diễn ra tại Client, component `CombatManager` ở Unity tự động ghi nhận mọi hành vi diễn ra của từng lượt đấu (Turn) vào một mảng nhật ký hành động cục bộ dưới dạng danh sách cấu trúc `CombatActionLog`.

Mỗi bản ghi hành động ghi lại chi tiết các thuộc tính:

- ID nhân vật tung chiêu và ID mục tiêu chịu đòn.
- Mã kỹ năng được kích hoạt.
- Lượng sát thương gây ra thực tế (Damage).
- Trạng thái sinh mệnh còn lại của mục tiêu.

b, Gửi yêu cầu xác thực qua gRPC (Validate PVE Result)

Khi trận đấu kết thúc (phe địch bị tiêu diệt hoàn toàn hoặc phe ta tử trận hết), Client (tại file `GameOverState.cs`) tiến hành đóng gói các thông tin liên quan và gửi một yêu cầu xác thực kết quả `ValidatePvEResultRequest` lên World Server thông qua API gRPC:

```
var req = new ValidatePvEResultRequest
{
    EnemyId = manager.EnemyID,
    IsVictory = allEnemiesDead,
    PlayerPower = manager.PlayerTotalPower,
    EnemyPower = manager.EnemyTotalPower,
};
req.CombatLogs.AddRange(manager.CombatLogs); // Nạp toàn bộ nhật
```

c, Thuật toán xác thực chéo (Cross-Validation Algorithm) tại Combat Server (Go)

Khi nhận được yêu cầu xác thực, vi dịch vụ `Combat Server` viết bằng Go (tại file `combat_service.go`) sẽ thực hiện thuật toán kiểm tra logic an toàn:

1. **Xử lý nhanh trường hợp thất bại:** Nếu Client báo cáo kết quả là thất bại (`IsVictory = false`), server chấp nhận ngay lập tức mà không cần xác thực để tiết kiệm CPU máy chủ.

2. Phân loại theo ngưỡng lực chiến:

- Nếu lực chiến của người chơi bình thường ($\text{PlayerPower} \geq 0.5 \times \text{EnemyPower}$), server chấp nhận kết quả chiến thắng.
- Nếu lực chiến của người chơi thấp bất thường ($\text{PlayerPower} < 0.5 \times \text{EnemyPower}$ - trường hợp nghi vấn hack chỉ số để vượt ải khó), server kích hoạt hàm xác thực nhật ký `verifyCombatLog`.

3. Xác thực cấu trúc nhật ký trận đấu: Hàm `verifyCombatLog` duyệt qua mảng `CombatLogs` gửi lên từ Client để kiểm tra hai ràng buộc logic:

- *Kiểm tra tính tồn tại*: Nếu danh sách log trống ($\text{len}(\text{logs}) == 0$) nhưng báo thắng → Xác định là hành vi hack (hủy kết quả).
- *Kiểm tra sát thương tối đa của một đòn đánh* (*Max Single Hit Damage Limit*): Sát thương của một đòn đánh đơn lẻ không được vượt quá giới hạn lý thuyết tính theo lực chiến hiện tại của người chơi:

$$\text{Damage}_{\text{hit}} \leq \text{MaxPossibleDamage} = \text{PlayerPower} \times 2$$

Nếu có bất kỳ dòng log nào chứa lượng sát thương lớn hơn giới hạn này → Xác định là hành vi hack chỉ số tấn công vô hạn (hủy kết quả).

- *Kiểm tra tổng sát thương gây ra*: Tổng sát thương gây ra bởi người chơi phải lớn hơn hoặc bằng lượng máu ước lượng tối thiểu của kẻ địch:

$$\sum \text{Damage}_{\text{hit}} \geq \text{EstimatedEnemyHP} = \frac{\text{EnemyPower}}{2}$$

Nếu tổng sát thương không đủ để tiêu diệt địch nhưng báo thắng → Đưa ra cảnh báo hệ thống.

d, Cập nhật tiến trình và trao thưởng tại World Server

Nếu kết quả xác thực hợp lệ (`IsValid = true`), World Server thực hiện lưu tiến trình vượt ải mới của người chơi vào PostgreSQL Game Shard DB, cộng điểm kinh nghiệm, tài nguyên gỗ/linh thạch thưởng vào tài sản người chơi và phản hồi thành công về Client. Nếu không hợp lệ, server trả về mã lỗi từ chối và ngăn chặn việc cộng thưởng.

5.3.3 Kết quả đạt được

Giải pháp kiểm thử và xác thực nhật ký trận đấu mang lại hiệu quả to lớn trong vận hành thực tế:

- ****Ngăn chặn gian lận hiệu quả****: Bảo vệ an toàn nền kinh tế game. Các nỗ

lực chỉnh sửa chỉ số tấn công hoặc gửi gói tin thắng giả mạo từ client đều bị server phát hiện và từ chối 100% do vi phạm các ràng buộc toán học về giới hạn sát thương tối đa.

- ****Tối ưu hóa hiệu năng máy chủ vượt trội****: Nhờ cơ chế chỉ xác thực các trường hợp chiến thắng nghi vấn (lực chiến thấp vượt ải khó) và kiểm tra duyệt mảng log trên bộ nhớ đệm RAM cực nhanh (không cần chạy lại tiến trình mô phỏng trận đấu), thời gian xử lý xác thực trên máy chủ chỉ mất dưới 1ms. Điều này giúp hệ thống backend chịu tải cực tốt, duy trì hoạt động ổn định với tỷ lệ lỗi request đạt mức 0.0% dưới tải đỉnh 5.000 CCU, và mức tiêu thụ RAM của Combat Server chỉ duy trì dưới 150MB, giúp tối ưu hóa chi phí vận hành phần cứng thực tế.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết lại toàn bộ kết quả nghiên cứu và sản phẩm thực tiễn của đồ án tốt nghiệp thiết kế và xây dựng hệ thống trò chơi trực tuyến nhiều người chơi “The First Sect Origin”. Đồng thời, chương này cũng phân tích chi tiết những đóng góp nổi bật, các bài học kinh nghiệm thu hoạch được trong suốt quá trình triển khai, nhận diện các mặt còn hạn chế của sản phẩm và đề xuất định hướng phát triển nhằm nâng cấp hệ thống trong tương lai.

6.1 Kết luận

Đồ án tốt nghiệp đã hoàn thành đầy đủ mục tiêu đề ra là thiết kế và xây dựng một hệ thống game thẻ tướng chiến thuật 2D hoàn chỉnh bao gồm phần máy khách (Client) chạy trên game engine Unity và hệ thống máy chủ (Backend) phân tán theo kiến trúc vi dịch vụ (Microservices) [1] viết bằng ngôn ngữ lập trình Go. Hệ thống đã chứng minh được tính khả thi, hiệu năng cao và khả năng bảo mật qua các bài kiểm thử nghiêm ngặt dưới tải đồng thời lớn.

6.1.1 So sánh với các nghiên cứu hoặc sản phẩm tương tự

Để làm rõ giá trị thực tiễn và tính tối ưu của giải pháp thiết kế mà đồ án lựa chọn, hệ thống máy chủ vi dịch vụ tự xây dựng được so sánh, đánh giá với hai hướng tiếp cận phổ biến hiện nay trong công nghiệp phát triển game: các dịch vụ máy chủ đám mây thương mại (Backend-as-a-Service - BaaS) như Microsoft PlayFab, Photon Engine và mô hình thiết kế hệ thống đơn khối truyền thống (Monolith) chạy trên nền Node.js kết hợp định dạng dữ liệu truyền tải JSON qua giao thức HTTP.

a, So sánh với các dịch vụ Backend đám mây thương mại (PlayFab, Photon Engine)

Các dịch vụ BaaS như Microsoft PlayFab hay Photon Engine thường được các studio game nhỏ lựa chọn nhờ ưu điểm triển khai nhanh chóng và giảm thiểu gánh nặng tự vận hành phần cứng ở giai đoạn khởi đầu. Tuy nhiên, khi so sánh với giải pháp tự phát triển của đồ án, các dịch vụ này bộc lộ những mặt hạn chế đáng kể:

- **Chi phí vận hành dài hạn:** Các dịch vụ đám mây thương mại tính phí dựa trên số lượng người dùng hoạt động hàng tháng (MAU) hoặc lượng người dùng trực tuyến đồng thời (CCU). Khi trò chơi phát triển quy mô và có lượng người chơi lớn, chi phí duy trì sẽ tăng theo cấp số nhân, gây ảnh hưởng trực tiếp đến hiệu quả tài chính của nhà phát hành khi quy mô trò chơi mở rộng. Ngược lại, hệ thống Go Microservices tự phát triển cho phép triển khai linh hoạt trên mọi hạ tầng máy chủ ảo (VPS) giá rẻ hoặc hệ thống máy chủ on-premise của doanh

ngành, giúp làm chủ chi phí vận hành cố định độc lập với sự tăng trưởng của CCU.

- **Độ trễ truyền thông (Network Latency):** Do máy chủ của PlayFab hay Photon đặt tại các trung tâm dữ liệu nước ngoài (Singapore, Hồng Kông, Nhật Bản, Mỹ) nên các gói tin truyền qua mạng internet quốc tế thường bị ảnh hưởng bởi băng thông tuyến cáp quang biển. Đối với thị trường Việt Nam, điều này gây ra độ trễ kết nối từ 80ms đến hơn 200ms. Hệ thống tự xây dựng trong đồ án sử dụng giao thức gRPC kết nối trực tiếp đến cụm server đặt tại Việt Nam, mang lại thời gian phản hồi thấp (dưới 15ms), tối ưu hóa trải nghiệm người dùng.
- **Khả năng tùy biến sâu và làm chủ mã nguồn:** Sử dụng PlayFab đồng nghĩa với việc lập trình viên phải chấp nhận các giới hạn nghiêm ngặt về mặt logic nghiệp vụ (Business Logic). Các hàm xử lý phía máy chủ (Cloud Script) bị cô lập trong môi trường runtime giới hạn thời gian chạy và dung lượng bộ nhớ. Lập trình viên không thể can thiệp sâu vào nhân cơ sở dữ liệu để thực hiện sharding, điều phối cơ chế cache động hoặc triển khai các thuật toán chống hack chiến đấu tùy biến. Với hệ thống tự xây dựng, đồ án làm chủ hoàn toàn mã nguồn, dễ dàng tích hợp các thuật toán xác thực chéo log chiến đấu dựa trên ngưỡng lực chiến, cơ chế ghi nhận tài nguyên lười Lazy Evaluation và cơ chế khóa phân tán Redis Mutex chống tương tranh dữ liệu.

b, So sánh với kiến trúc máy chủ đơn khối truyền thông (Node.js Monolith + JSON)

Kiến trúc đơn khối (Monolith) kết hợp Node.js/Express và truyền dữ liệu JSON qua giao thức HTTP/1.1 là giải pháp rất phổ biến do dễ lập trình và cộng đồng hỗ trợ lớn. Tuy nhiên, khi hệ thống đòi hỏi khả năng xử lý tương tranh cao và tối ưu hóa tài nguyên phần cứng, kiến trúc này bộc lộ nhiều điểm nghẽn:

- **Hiệu năng xử lý tương tranh (Concurrency):** Node.js vận hành dựa trên cơ chế đơn luồng (Single-Threaded Event Loop). Khi gặp các tác vụ yêu cầu tính toán CPU nặng (chẳng hạn như phân tích cú pháp chuỗi log chiến đấu dài hoặc tính toán ma trận thuộc tính nhân vật), Node.js dễ bị nghẽn xử lý (CPU-bound blocking), làm chậm toàn bộ các yêu cầu khác đang chờ trong hàng đợi. Trong khi đó, ngôn ngữ Go của hệ thống tự xây dựng hỗ trợ tương tranh mức phần cứng thông qua cơ chế Goroutine siêu nhẹ (chỉ tốn khoảng 2KB RAM khởi tạo so với 1-2MB của hệ điều hành Thread). Go có khả năng tận dụng tối đa kiến trúc CPU đa nhân để xử lý đồng thời hàng chục vạn kết nối mà không gây nghẽn hệ thống.

- **Hiệu năng truyền tải và định dạng dữ liệu:** Định dạng JSON truyền qua HTTP/1.1 là định dạng văn bản (text-based), đòi hỏi kích thước gói tin lớn và tiêu tốn nhiều tài nguyên CPU của cả Client lẫn Server cho quá trình mã hóa/giải mã (Serialization/Deserialization). Hệ thống của đồ án sử dụng gRPC truyền tải dữ liệu nhị phân (binary-based) được tuần tự hóa cực nhanh bằng Protocol Buffers trên nền giao thức HTTP/2 (hỗ trợ ghép kênh Multiplexing). Kích thước các gói tin truyền thông trong game (như Battle Log, thông tin nhân vật) giảm từ 60% đến 80%, giúp tiết kiệm đáng kể băng thông đường truyền và giảm tải xử lý cho CPU.
- **Khả năng mở rộng độc lập (Scalability):** Trong kiến trúc đơn khối, khi một chức năng (ví dụ như mô phỏng và xác thực trận đấu của Combat Server) bị quá tải CPU, lập trình viên bắt buộc phải nhân bản toàn bộ mã nguồn của cả hệ thống Monolith lên các máy chủ mới, gây lãng phí tài nguyên RAM và Database connections cho các thành phần không cần thiết khác (như Login, Chat). Với kiến trúc vi dịch vụ Go của đồ án, hệ thống được chia nhỏ thành các thành phần độc lập (Gateway, Login, World, Combat). Nhờ đó, ta có thể triển khai co giãn độc lập dịch vụ Combat Server lên hàng chục pod máy chủ khác để chịu tải chiến đấu, trong khi giữ nguyên số lượng pod của Login Server hay World Server, tối ưu hóa triệt để tài nguyên phần cứng đám mây.

Để tổng hợp lại các so sánh trên một cách trực quan, Bảng 6.1 thể hiện các tiêu chí kỹ thuật cốt lõi giữa hệ thống tự xây dựng của đồ án với hai hướng tiếp cận phổ biến trong công nghiệp.

Tiêu chí so sánh	Hệ thống vi dịch vụ Go + gRPC (Đồ án)	Dịch vụ đám mây BaaS (PlayFab / Photon)	Kiến trúc đơn khối Node.js + JSON
Chi phí vận hành	Thấp, cố định theo cấu hình VPS thực tế.	Tăng theo cấp số nhân dựa trên MAU/CCU.	Trung bình, tốn phần cứng do hiệu năng CPU kém.
Độ trễ kết nối (Latency)	Cực thấp (<15ms) do đặt server nội địa.	Cao (80ms - 200ms) do datacenter ở nước ngoài.	Trung bình đến cao (30ms - 80ms) do dùng HTTP/1.1.
Xử lý đồng thời (Concurrency)	Rất cao (hơn 10^6 Goroutines đồng thời trên máy chủ từ 8 GB RAM).	Cao, phụ thuộc vào hạ tầng đám mây thuê ngoài.	Trung bình, bị giới hạn bởi cơ chế đơn luồng.
Khả năng tùy biến logic	Tuyệt đối (100% mã nguồn tự viết).	Bị hạn chế bởi API đóng gói và Cloud Script.	Cao (tự viết mã nguồn) nhưng khó tách biệt dịch vụ.
Bảo mật và chống hack	Tích hợp xác thực chéo log chiến đấu thời gian thực.	Phụ thuộc vào các bộ lọc cơ bản hoặc Cloud Script giới hạn tải.	Khó triển khai xác thực chéo hiệu năng cao do nghẽn CPU.
Băng thông truyền tải	Cực thấp (tuần tự hóa nhị phân Protocol).	Trung bình, phụ thuộc giao thức API của BaaS.	Cao, sử dụng định dạng văn bản JSON qua HTTP.

Bảng 6.1: Bảng so sánh tổng quan giải pháp công nghệ của đồ án với các sản phẩm tương tự

6.1.2 Phân tích quá trình thực hiện đồ án tốt nghiệp

Trải qua quá trình nghiên cứu lý thuyết, thiết kế hệ thống và phát triển thực nghiệm, đồ án đã đạt được các kết quả quan trọng, đồng thời đánh giá khách quan các hạn chế còn tồn tại cần được khắc phục để hoàn thiện sản phẩm.

a, Những kết quả đã đạt được

Đồ án đã hiện thực hóa thành công sản phẩm trò chơi thẻ tướng chiến thuật 2D “The First Sect Origin” bám sát các mục tiêu đề ra ban đầu, đạt được các kết quả cụ thể sau:

- 1. Phát triển hoàn chỉnh Game Client (Unity):** Xây dựng ứng dụng máy khách trên nền tảng Unity 6 với hệ thống giao diện thích ứng (Responsive UI) chạy tối ưu ở độ phân giải thiết kế 2778×1284 và mật độ điểm ảnh 240 DPI, hoạt động trơn tru trên mọi tỷ lệ màn hình di động Android khác nhau. Tích hợp công nghệ phông chữ vector SDF (Signed Distance Field) sắc nét ở mọi độ phân giải. Xây dựng bộ phân tích cú pháp nhật ký trận đấu (Battle Log parser)

để phát lại (Replay) diễn biến trận đấu sinh động dưới dạng hoạt ảnh 2D, hiệu ứng kỹ năng và âm thanh sống động.

2. **Thiết kế hệ thống máy chủ vi dịch vụ (Go):** Xây dựng thành công hệ thống Backend phân tán gồm 4 dịch vụ độc lập giao tiếp qua gRPC nhị phân hiệu năng cao: Gateway Server (định tuyến, bảo mật), Login Server (xác thực, mã hóa Bcrypt, cấp khóa token JWT), World Server (quản lý người chơi, công trình, tài nguyên), và Combat Server (mô phỏng, xác thực trận đấu phi trạng thái).
3. **Bảo mật dữ liệu và giải quyết tương tranh phân tán:** Ứng dụng thành công cơ chế khóa phân tán Redis Mutex chống Race Condition khi người chơi spam request thu hoạch tài nguyên đồng thời từ nhiều thiết bị. Tích hợp giải pháp kiểm soát tương tranh lạc quan OCC tại cơ sở dữ liệu PostgreSQL để bảo vệ an toàn tuyệt đối cho số dư tài sản của người dùng.
4. **Triển khai DevOps và Kiểm thử tải:** Đóng gói toàn bộ ứng dụng bằng Docker và điều phối qua Docker Compose. Sử dụng Nginx Reverse Proxy làm công cụ định tuyến đầu vào và cấp chứng chỉ bảo mật. Tiến hành kiểm thử tải trọng (Load Testing) giả lập thành công kịch bản 5.000 CCU hoạt động đồng thời; hệ thống máy chủ vận hành ổn định và xử lý thành công toàn bộ các yêu cầu truyền đến với độ trễ phản hồi trung bình cực thấp (5 ms) và mức tiêu hao tài nguyên phần cứng tối ưu.

b, Những hạn chế và việc chưa làm được

Bên cạnh những kết quả tích cực, hệ thống vẫn tồn tại một số điểm hạn chế cần được khắc phục trong các phiên bản tiếp theo:

- **Hạ tầng Redis chưa phân tán (Single Node):** Hệ thống cache và quản lý khóa phân tán Redis hiện tại mới chỉ chạy trên một thực thể độc lập (Single Node). Điều này tạo ra điểm nghẽn chịu lỗi duy nhất (Single Point of Failure - SPoF). Nếu máy chủ Redis gặp sự cố sập nguồn hoặc quá tải vật lý, toàn bộ cơ chế khóa phân tán và lưu trữ phiên hoạt động của người chơi sẽ bị gián đoạn.
- **Chưa tự động hóa Sharding cơ sở dữ liệu:** Cơ chế phân vùng cơ sở dữ liệu PostgreSQL hiện tại mới chỉ được thiết kế ở mức logic trên mã nguồn World Server dựa trên ID người chơi, chưa được cấu hình tự động phân chia vật lý (Dynamic Sharding) trên cụm cơ sở dữ liệu thực tế và chưa tích hợp khả năng co giãn tự động đồng bộ với Kubernetes.
- **Chưa hỗ trợ tính năng đấu trường đối kháng (PvP):** Hệ thống hiện tại mới chỉ tập trung phát triển và mô phỏng chế độ chơi vượt ải cốt truyện (PvE)

đánh với quái vật do máy tính điều khiển. Trò chơi chưa xây dựng được tính năng đấu trường đối kháng giữa người chơi với người chơi (PvP), bao gồm cả hình thức đối kháng không đồng bộ (Asynchronous PvP - thách đấu đội hình phòng thủ lưu sẵn của người chơi khác do máy tính điều khiển) lẫn hình thức đối kháng trực tuyến thời gian thực (Real-time PvP).

- **Hệ thống kỹ năng và chiều sâu gameplay còn đơn giản:** Hệ thống kỹ năng chiến thuật của các đệ tử tông môn mới dừng lại ở các hiệu ứng cơ bản như gây sát thương vật lý, sát thương phép thuật, và hồi phục máu. Hệ thống chưa có các hiệu ứng chiến thuật phức tạp (khống chế cứng, cướp lượt, hoặc tương tác môi trường chiến đấu).

c, Các đóng góp nổi bật của đề án tốt nghiệp

Đóng góp lớn nhất của đề án tốt nghiệp nằm ở việc giải quyết thành công hai bài toán kinh điển trong phát triển game online nhiều người chơi bằng các giải pháp công nghệ tối ưu:

1. **Giải pháp xác thực chéo nhật ký trận đấu (Combat Log Validation) hiệu năng cao chống hack phía Client:** Thay vì để máy chủ phải chạy lại toàn bộ tiến trình mô phỏng trận đấu từ đầu gây quá tải CPU nghiêm trọng, hoặc tin tưởng hoàn toàn vào kết quả của Client dẫn đến lỗ hổng hack chỉ số, đề án đã đề xuất giải pháp xác thực chéo phi trạng thái trên RAM. Bằng việc áp dụng các ràng buộc toán học thông minh (ngưỡng lực chiến tương quan, giới hạn sát thương tối đa của một đòn đánh theo công thức $\text{Damage}_{\text{hit}} \leq \text{PlayerPower} \times 2$, và kiểm tra tính tồn tại của log), Combat Server viết bằng Go có thể xác thực tính đúng đắn của trận đấu chỉ trong chưa đầy 1ms, chặn đứng 100% các nỗ lực chỉnh sửa bộ nhớ client hay gửi gói tin thắng giả mạo mà vẫn tối ưu hóa chi phí phần cứng.
2. **Cơ chế đồng bộ hóa tài nguyên nhàn rỗi lười (Lazy Evaluation):** Đề án đã loại bỏ hoàn toàn các tiến trình chạy ngầm ghi cơ sở dữ liệu định kỳ cho hàng vạn tài khoản (vốn là nguyên nhân hàng đầu gây nghẽn I/O ghi tại Database). Bằng thuật toán tính toán động mức thời gian chênh lệch chỉ khi có yêu cầu thực tế phát sinh, lượng câu lệnh cập nhật DB giảm từ hàng ngàn câu lệnh mỗi giờ xuống còn 2-3 câu lệnh thực tế cho mỗi người chơi hoạt động, giúp CPU của PostgreSQL luôn duy trì dưới mức 15% ngay cả khi chạy giả lập tải đỉnh 5.000 CCU.

d, Bài học kinh nghiệm rút ra

Quá trình thực hiện đề án đã đúc rút được nhiều kinh nghiệm thực tiễn quan trọng về mặt chuyên môn và thiết kế hệ thống:

- **Tư duy thiết kế hệ thống phân tán:** Hiểu rõ cách phân rã một ứng dụng lớn thành các vi dịch vụ độc lập, thiết kế giao thức giao tiếp gRPC hiệu quả và cách tổ chức luồng đi của dữ liệu qua Gateway.
- **Kỹ năng tối ưu hóa hiệu năng:** Nhận thức được rằng một hệ thống tốt không chỉ chạy đúng mà phải chạy hiệu quả. Việc lựa chọn công nghệ (Go, gRPC, Redis) và thiết kế giải thuật thông minh (Lazy Evaluation, stateless combat simulation) quyết định sự sống còn của hệ thống dưới tải lớn.
- **Kỹ năng xử lý tương tranh dữ liệu thực tế:** Nắm vững cách thức áp dụng khóa phân tán Redis Mutex và kiểm soát tương tranh lạc quan để giải quyết triệt để các bài toán Race Condition trong môi trường phân tán – điều mà lý thuyết sách vở rất khó truyền tải hết tính phức tạp.
- **Kỹ năng làm việc nhóm và quy trình DevOps:** Học cách phối hợp công việc thông qua Git, tự động hóa đóng gói, triển khai ứng dụng bằng Docker/Nginx, và tư duy kiểm thử tải thực tế để đánh giá chất lượng sản phẩm một cách khách quan.

6.2 Hướng phát triển

Để hoàn thiện sản phẩm game “The First Sect Origin” và đưa dự án tiếp cận gần hơn với tiêu chuẩn thương mại hóa thực tế, định hướng công việc trong tương lai được nhóm nghiên cứu chia làm hai giai đoạn rõ ràng: hoàn thiện các tính năng/nhiệm vụ hiện tại và nghiên cứu nâng cấp các hướng đi công nghệ mới.

6.2.1 Các công việc cần thiết để hoàn thiện sản phẩm

Trước tiên, hệ thống cần tập trung giải quyết các điểm hạn chế đã được nhận diện ở phiên bản hiện tại nhằm đảm bảo độ tin cậy, tính an toàn và chiều sâu của trò chơi:

1. **Tích hợp chữ ký số bảo mật cho gói tin Battle Log:** Để nâng cấp lớp bảo vệ an ninh thông tin, ngăn chặn hacker phân tích cấu trúc Protobuf để xây dựng các chương trình giả lập log chiến đấu hợp lệ vượt qua bộ lọc của Combat Server, hệ thống sẽ áp dụng cơ chế ký số điện tử (Digital Signature) sử dụng mã hóa bất đối xứng (như ECDSA - Elliptic Curve Digital Signature Algorithm).

Quy trình hoạt động đề xuất: Khi người chơi bắt đầu trận đấu, Login/World Server sẽ cấp cho Client một cặp khóa phiên (session key) ngắn hạn hoặc sử dụng khóa bí mật được sinh ngẫu nhiên lưu trong vùng nhớ an toàn của thiết bị (Android Keystore hoặc iOS Keychain). Khi trận đấu kết thúc, Client thực hiện băm dữ liệu Battle Log và dùng khóa bí mật (Private Key) để ký số lên

gói tin. Combat Server khi nhận được gói tin sẽ dùng khóa công khai (Public Key) tương ứng để xác thực chữ ký. Nếu gói tin bị chỉnh sửa dù chỉ một bit trên đường truyền, chữ ký sẽ lập tức bị coi là bất hợp lệ và yêu cầu xác thực bị hủy bỏ ngay lập tức.

2. **Triển khai Redis Cluster và PostgreSQL Sharding vật lý:** Để loại bỏ điểm nghẽn chịu lỗi duy nhất SPoF của bộ nhớ đệm, hệ thống sẽ cấu hình cụm Redis Cluster phân tán gồm ít nhất 3 node Master và 3 node Slave. Dữ liệu khóa phân tán và phiên làm việc sẽ được phân chia tự động qua 16384 hash slot của Redis, đảm bảo tính sẵn sàng cao (High Availability). Nếu một node Master bị sập, node Slave tương ứng sẽ tự động được nâng cấp lên làm Master trong vòng vài giây mà không gây gián đoạn hệ thống.

Song song đó, cơ sở dữ liệu PostgreSQL sẽ được cấu hình sharding vật lý thực tế bằng công cụ trung gian (như Citus hoặc các bộ router sharding chuyên dụng), giúp phân tán dữ liệu người chơi sang nhiều máy chủ database vật lý khác nhau, tăng giới hạn đọc/ghi và giảm tải cho một máy chủ database duy nhất.

3. **Mở rộng hệ thống kỹ năng đệ tử đa dạng:** Nâng cấp mã nguồn mô phỏng chiến đấu tại cả Client (Unity) và Server (Go Combat Server) để hỗ trợ các hiệu ứng trạng thái (Status Effects / Buffs / Debuffs) phức tạp:
 - **Khống chế lượt đi:** Các kỹ năng gây choáng (Stun), đóng băng (Freeze) khiến mục tiêu mất lượt, hoặc kỹ năng tăng tốc độ đánh (Speed Buff) giúp đệ tử được ưu tiên tấn công trước.
 - **Hút máu và Phản sát thương:** Kỹ năng chuyển hóa phần trăm sát thương gây ra thành lượng máu hồi phục cho bản thân (Lifesteal), hoặc phản lại một phần sát thương nhận vào cho kẻ tấn công (Thorns).
 - **Triệu hồi và Hiệu ứng môi trường:** Cho phép đệ tử triệu hồi thêm các vật thể phụ trợ (như con rối, thú triệu hồi) tham gia chiến đấu, hoặc tạo ra các trận pháp thay đổi thuộc tính của toàn bộ nhân vật trên sân đấu (như giảm công vật lý, tăng thủ phép).

6.2.2 Định hướng phát triển và nâng cấp mới

Sau khi hoàn thiện các tính năng hiện hữu, các hướng đi mới mang tính đột phá về mặt công nghệ và trải nghiệm người dùng sẽ được nghiên cứu và tích hợp vào hệ thống:

a, Xây dựng tính năng đấu trường đối kháng (PvP)

Để mang lại tính tương tác xã hội cao và tăng sức hấp dẫn cho trò chơi, đồ án định hướng phát triển một dịch vụ vi dịch vụ mới chuyên trách quản lý đấu trường đối kháng PvP, bao gồm chế độ PvP không đồng bộ và đối kháng trực tuyến thời gian thực:

- **Đối kháng không đồng bộ (Asynchronous PvP):** Xây dựng cơ chế cho phép người chơi thách đấu với đội hình phòng thủ do AI điều khiển của người chơi khác. World Server sẽ truy vấn dữ liệu đội hình và thuộc tính nhân vật đã lưu của đối thủ từ PostgreSQL, chuyển dữ liệu sang Combat Server để thực hiện mô phỏng phi trạng thái tương tự như chế độ PvE và trả về nhật ký trận đấu.
- **Đối kháng trực tiếp thời gian thực (Real-time PvP):** Sử dụng tính năng truyền phát hai chiều (*Bidirectional Streaming*) của gRPC trên nền HTTP/2 hoặc giao thức WebSocket [8] để duy trì luồng dữ liệu liên tục giữa hai client và PvP Server, đồng bộ các thao tác chiến đấu với độ trễ dưới 20ms.
- **Cơ chế đồng bộ hóa trạng thái (State Synchronization):** Áp dụng mô hình máy chủ làm trọng tài (Server-Authoritative) kết hợp kỹ thuật dự đoán phía client (Client-side Prediction) và bù trừ độ trễ mạng (Lag Compensation) để đảm bảo trận đấu Real-time PvP diễn ra mượt mà ngay cả trong điều kiện mạng chập chờn.

b, Triển khai hệ thống tự động co giãn (Auto-scaling) trên Kubernetes (K8s)

Để đáp ứng linh hoạt với lưu lượng người chơi thay đổi liên tục trong thực tế (ví dụ: lượng người chơi tăng đột biến vào các khung giờ sự kiện hoặc tối cuối tuần), hệ thống sẽ được chuyển đổi hạ tầng triển khai sang cụm Kubernetes:

- **Đóng gói và quản lý Pod:** Mỗi vi dịch vụ (Gateway, Login, World, Combat) sẽ được định nghĩa trong các tệp tin cấu hình Kubernetes Deployment độc lập, cho phép quản lý vòng đời và cấu hình giới hạn tài nguyên (CPU/RAM Requests và Limits) cho từng container một cách chi tiết.
- **Tích hợp Horizontal Pod Autoscaler (HPA):** Cấu hình HPA để tự động theo dõi các chỉ số hiệu năng hệ thống thông qua Prometheus và Grafana.

Khi mức tiêu thụ CPU trung bình của các pod Combat Server vượt quá ngưỡng 70%, hoặc khi số lượng kết nối gRPC đồng thời tăng cao, Kubernetes sẽ tự động sinh thêm (scale up) các pod Combat Server mới trong vòng vài giây để san sẻ tải lượng tính toán chiến đấu. Ngược lại, khi lượng người chơi giảm xuống vào ban đêm, hệ thống sẽ tự động giải phóng (scale down) các pod dư

thừa để tiết kiệm chi phí thuê tài nguyên đám mây cho nhà phát hành.

- **Quản lý định tuyến động qua Ingress Controller:** Sử dụng các công cụ Ingress Controller hiện đại (như Nginx Ingress Controller) để tự động phát hiện và định tuyến các yêu cầu kết nối từ Gateway Server đến các pod vi dịch vụ mới được khởi tạo một cách trơn tru, không gây gián đoạn bất kỳ phiên kết nối nào của người chơi đang trực tuyến.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

TÀI LIỆU THAM KHẢO

- [1] S. Newman, *Building Microservices*. O'Reilly Media, 2021.
- [2] Unity Technologies. "Unity user manual," Accessed: May 18, 2026. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [3] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Addison-Wesley Professional, 2015.
- [4] K. Indrasiri and D. Kuruppu, *gRPC: Up and Running*. O'Reilly Media, 2020.
- [5] PostgreSQL Global Development Group. "Postgresql 16 documentation," Accessed: Apr. 12, 2026. [Online]. Available: <https://www.postgresql.org/docs/16/index.html>
- [6] J. L. Carlson, *Redis in Action*. Manning Publications, 2013.
- [7] M. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," Internet Engineering Task Force, RFC 7519, 2015.
- [8] I. Fette and A. Melnikov, "The websocket protocol," Internet Engineering Task Force, RFC 6455, 2011.

PHỤ LỤC

A. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

A.1 Đặc tả use case Đăng ký tài khoản

- **Tên use case:** Đăng ký tài khoản.
- **Tác nhân:** Người chơi.
- **Tiền điều kiện:** Người chơi đã tải ứng dụng Client, thiết bị có kết nối mạng Internet và chưa đăng ký tài khoản (hoặc muốn tạo thêm tài khoản mới).
- **Hậu điều kiện:** Tài khoản mới được ghi nhận thành công vào cơ sở dữ liệu PostgreSQL của hệ thống, mật khẩu được mã hóa an toàn, người chơi sẵn sàng đăng nhập vào game.
- **Luồng sự kiện chính:**
 1. Người chơi chọn chức năng "Đăng ký" trên màn hình chính của Client.
 2. Người chơi điền thông tin tài khoản bao gồm: tên đăng nhập (username), mật khẩu (password) và địa chỉ email.
 3. Client gửi yêu cầu đăng ký kèm dữ liệu tài khoản đến dịch vụ Login.
 4. Dịch vụ Login thực hiện kiểm tra tính hợp lệ của dữ liệu đầu vào (tên tài khoản chưa tồn tại, độ dài mật khẩu tối thiểu 6 ký tự, email đúng định dạng).
 5. Hệ thống thực hiện mã hóa mật khẩu sử dụng thuật toán bcrypt, tạo bản ghi tài khoản mới trong cơ sở dữ liệu PostgreSQL.
 6. Dịch vụ Login trả về kết quả đăng ký thành công cho Client.
 7. Client hiển thị thông báo thành công và chuyển hướng người chơi về màn hình đăng nhập.
- **Luồng sự kiện phát sinh:**
 - *Tên đăng nhập đã tồn tại:* Dịch vụ Login kiểm tra thấy username đã có người sử dụng, phản hồi mã lỗi tương ứng. Client nhận thông tin lỗi và yêu cầu người chơi chọn tên đăng nhập khác.
 - *Định dạng thông tin sai:* Mật khẩu quá ngắn hoặc email không hợp lệ, hệ thống từ chối đăng ký và trả về thông tin lỗi tương ứng để người chơi điều chỉnh.

A.2 Đặc tả use case Sắp xếp đội hình chiến đấu

- **Tên use case:** Sắp xếp đội hình chiến đấu.
- **Tác nhân:** Người chơi.
- **Tiền điều kiện:** Người chơi đã đăng nhập vào game thành công và sở hữu ít nhất một đệ tử trong danh sách đệ tử tông môn.
- **Hậu điều kiện:** Cấu hình đội hình chiến đấu mới của người chơi được cập nhật vào cơ sở dữ liệu và được sử dụng khi tham gia vượt ải chiến dịch.
- **Luồng sự kiện chính:**
 1. Người chơi truy cập vào giao diện "Đội hình" từ màn hình chính tông môn.
 2. Hệ thống hiển thị danh sách đệ tử hiện có và sơ đồ trận hình hiện tại.
 3. Người chơi kéo thả hoặc click chọn đệ tử từ danh sách vào các vị trí trong sơ đồ trận hình (các vị trí hàng trước và hàng sau).
 4. Người chơi bấm nút "Xác nhận" để lưu lại sơ đồ đội hình mới.
 5. Client gửi yêu cầu thay đổi đội hình kèm danh sách ID đệ tử và mã vị trí tương ứng qua Gateway tới dịch vụ World.
 6. Dịch vụ World kiểm tra tính hợp lệ của yêu cầu (đệ tử có thực sự thuộc sở hữu của người chơi, số lượng đệ tử không vượt quá giới hạn tối đa của đội hình).
 7. Dịch vụ World thực hiện cập nhật cấu hình trận hình của người chơi trong cơ sở dữ liệu PostgreSQL.
 8. Dịch vụ World phản hồi lưu thành công về Client.
 9. Client cập nhật giao diện hiển thị đội hình mới nhất.
- **Luồng sự kiện phát sinh:**
 - *Đệ tử không hợp lệ:* ID đệ tử không tồn tại hoặc không nằm trong danh sách sở hữu của người chơi, hệ thống hủy bỏ giao dịch, báo lỗi về Client và giữ nguyên đội hình cũ.

A.3 Đặc tả use case Thu thập tài nguyên công trình

- **Tên use case:** Thu thập tài nguyên công trình.
- **Tác nhân:** Người chơi.
- **Tiền điều kiện:** Người chơi đã đăng nhập, tông môn sở hữu các công trình sản

xuất tài nguyên nhân rồi (như Nhà Gỗ, Mỏ Đá, Linh Thạch Khoáng) và các công trình này đã tích lũy được lượng sản lượng lớn hơn 0 kể từ lần thu hoạch trước.

- **Hậu điều kiện:** Lượng tài nguyên nhân rồi tích lũy trong các công trình được chuyển vào ví tài sản của người chơi (Gỗ, Đá, Linh thạch), trạng thái tích lũy của công trình được đưa về 0.

- **Luồng sự kiện chính:**

1. Người chơi click vào biểu tượng tài nguyên hiển thị phía trên công trình sản xuất tài nguyên ở bản đồ tông môn.
2. Client gửi yêu cầu thu thập tài nguyên kèm ID công trình qua Gateway tới dịch vụ World.
3. Dịch vụ World truy vấn cơ sở dữ liệu để lấy mốc thời gian thu hoạch gần nhất của công trình, tính toán sản lượng tích lũy dựa trên công suất sản xuất của công trình và thời gian đã trôi qua.
4. Dịch vụ World cập nhật lượng tài nguyên tích lũy của công trình về 0 và cộng lượng tài nguyên đã thu hoạch vào tổng số dư tài sản (Gỗ, Đá, Linh thạch) của người chơi trong cơ sở dữ liệu PostgreSQL.
5. Dịch vụ World gửi phản hồi thành công kèm lượng tài nguyên thu hoạch được và số dư tài sản mới về Client.
6. Client thực hiện hoạt ảnh bay tài nguyên và cập nhật số lượng tài nguyên mới trên giao diện người chơi.

- **Luồng sự kiện phát sinh:**

- *Lượng tài nguyên bằng 0:* Thời gian giữa hai lần thu hoạch quá ngắn dẫn đến lượng tài nguyên sản sinh bằng 0, hệ thống từ chối yêu cầu thu hoạch hoặc trả về sản lượng bằng 0.